

Hybrid PQ-OPAQUE: A Three-Message Augmented PAKE from OPAQUE and ML-KEM-768

Oleksandr Melnychenko

Khmelnytskyi National University, Khmelnytskyi, Ukraine

Abstract

Password-backed accounts concentrate security risk in a human-memorable secret that may protect private state, authorization decisions, recovery flows, and audit-relevant actions for years. In a conventional architecture, that secret is often sent across the service boundary for verification. TLS is indispensable on the network, but its guarantee ends at the termination point. Beyond that point, application bugs, memory disclosure, logging paths, backups, replicated stores, and credential database compromise can turn password material into an offline guessing surface. Password-authenticated key exchange changes this interface by using the password to prove knowledge and derive fresh key material, rather than delivering a reusable credential to the service. For account systems, the augmented form of PAKE is the natural fit because the record-holding side should authenticate users without keeping a direct password verifier. OPAQUE is a mature realization of this augmented model. Its responder stores an authentication record instead of a password-equivalent secret, which reduces the value of database disclosure when the OPRF root secret remains outside the compromised record store. OPAQUE is therefore a strong baseline here, but its protection against future quantum adversaries is still limited by the classical key-exchange layer. The authenticated transcript is derived from elliptic-curve material and carries no native post-quantum secret.

This work presents Hybrid PQ-OPAQUE, an OPAQUE-derived augmented PAKE that adds ephemeral ML-KEM-768 material to the KE1–KE2–KE3 exchange without introducing a fourth message.

The construction combines four Ristretto255 Diffie–Hellman values with the ML-KEM shared secret by means of HKDF under a transcript-derived salt. Its specification fixes byte-level message layouts, version handling, and

account-context binding through `acct`. The authenticated transcript includes both pk_{kem} and ct_{kem} , while server-side OPRF keys are derived per account from the OPRF root secret σ_{oprf} .

The design is evaluated with ProVerif 2.05 models for secrecy and authentication, a Tamarin Prover 1.10.0 surrogate 4DH model, and a Rust artifact covering serialization, rejection behavior, state lifetime, and benchmark paths. These checks support the stated design boundary, but do not constitute a full computational proof of the hybrid combiner or a machine-checked proof of implementation correctness.

On the stated Linux/Xeon benchmark platform, the complete hybrid handshake occupies 2715 bytes in wire format, end-to-end authentication takes 836.11 ms, and the post-quantum path without the key-stretching function adds 379.45 μs . The measurements show that Argon2id dominates latency, while ML-KEM mainly affects transport size. The main operational boundary is separation of σ_{oprf} from passive authentication records.

Keywords: OPAQUE, PAKE, ML-KEM-768, post-quantum cryptography, hybrid key exchange, Ristretto255, HKDF, Tamarin, ProVerif, Rust

1. Introduction

1.1. Motivation

Passwords still carry most of the authentication burden in deployed systems. Across web applications and network services, passwords remain the dominant authentication mechanism [1, 2], and their position has proved stubbornly resistant to replacement. The cryptographic response to this reality is password-authenticated key exchange (PAKE), a family of protocols in which two parties who share only a low-entropy secret can establish a protected channel without ever exposing that secret to an adversary [3]. For real deployments the augmented variants matter most, because they ask what survives once the server itself is breached. OPAQUE [4] is the most thoroughly studied answer. When only the authentication database leaks, the server holds a cryptographic record rather than anything equivalent to the password [5]. That guarantee is not unconditional. It rests on a trust boundary many descriptions leave implicit, because losing the global secret from which the OPRF keys are derived is a different and far more damaging event than losing a passive copy of the record store.

The assumptions underneath this guarantee are classical, and that is exactly what a quantum adversary undermines. Shor’s algorithm breaks the discrete-logarithm problem on which elliptic-curve key exchange, and the PAKE constructions built over it, ultimately depend [6, 7]. How near that threat stands is still a moving target. An earlier estimate placed the cost of breaking Ristretto255/Curve25519 at roughly 2330 logical qubits [8], whereas a more recent analysis of 256-bit ECDLP over the `secp256k1` curve reports either no more than 1200 logical qubits with 90 million Toffoli gates or no more than 1450 with 70 million [9]. Neither figure is parameter-specific to Ristretto255, and we invoke them only to fix a contemporary sense of scale for 256-bit migration risk. What turns this from a distant worry into a present one is the harvest-now-decrypt-later strategy [10]. An adversary who records today’s traffic and simply waits can still profit once the hardware arrives, which is the precise predicament of any secret meant to stay confidential for years.

Standardization has started to close this gap, though unevenly. In August 2024 NIST issued its first post-quantum standards, FIPS 203 for ML-KEM and FIPS 204 for ML-DSA [11, 12], and hybrid designs that pair a classical exchange with a post-quantum one already appear in TLS 1.3 [13], in Signal’s PQXDH [15], building on the post-quantum X3DH analysis of Brendel et al. [14], and in WireGuard [16]. Password authentication has not kept pace. The current IRTF specification of OPAQUE [5] offers nothing against a quantum attacker, and the same holds for established alternatives such as SRP and SPAKE2. The research literature is further along, with compact KEM-based constructions [17], schemes carrying UC and quantum random oracle model (QROM) guarantees [18], and dedicated analyses of hybrid composition [19, 20]. What remains hard to assemble is the combination a practitioner migrating an augmented PAKE actually needs, namely an explicit message syntax, a claim set whose limits are stated rather than assumed, concrete measurements, and evidence from a running implementation. Those ingredients rarely arrive together.

Where final standards exist, we build on them directly. ML-KEM-768 follows FIPS 203 [11], the OPAQUE baseline is read against RFC 9807 [5], Ristretto255 follows RFC 9496 [28], and HKDF follows RFC 5869 [33]. Argon2id [32] provides the password-hardening baseline, and Criterion [37] serves only to take the measurements reported later.

A second motivation comes from a less obvious setting. In audit-heavy cyber-physical systems a login is seldom an isolated web session. More often

it is one event in a longer chain of telemetry, inspection, and operational control [38, 39, 40]. Such systems reward protocols with stable, recordable transcripts and unambiguous failure behavior, and that expectation informs several of the design choices below. These applications motivate the engineering, but they do not by themselves vouch for the cryptographic security of Hybrid PQ-OPAQUE.

1.2. Research Scope and Evaluation Objective

This paper studies a narrower engineering question. We ask whether OPAQUE can be reinforced with ML-KEM-768 while preserving its three-message authentication flow. The construction combines four Ristretto255 Diffie–Hellman values with an ML-KEM-768 shared secret, fixes wire-format versioning, and binds an explicit account context represented by the account identifier `acct` into the transcript. We evaluate the result using symbolic analysis and an executable reference implementation in Rust. What we offer is not a general theory of hybrid PAKE composition. It is an implementation-level augmented-PAKE design with explicit syntax, stated assumptions, and a narrowly limited claim set.

Our evaluation objective is to determine whether this migration profile can be specified, modeled, implemented, and measured without silently changing the OPAQUE interaction profile or the augmented-PAKE trust boundary. The paper therefore keeps each claim tied to a corresponding evidence layer, as summarized in Table 1. Throughout, the term research artifact refers to the accompanying source package that holds these evidence layers in alignment, namely the Rust crates, formal models, archived verification logs, tests, benchmark harnesses, and adapter documentation.

1.3. Research Contributions

1. **A concrete OPAQUE migration profile.** The paper specifies a three-message OPAQUE-derived construction that combines four Ristretto255 DH terms with an ML-KEM-768 shared secret, fixes wire-format versioning, and binds account context, pk_{kem} , and ct_{kem} into the authenticated transcript.
2. **A scoped evidence package.** The manuscript aligns reduction sketches, two ProVerif models, a surrogate Tamarin 4DH model, and executable conformance tests to the same claim boundary, while explicitly separating symbolic evidence from implementation proof.

Table 1: Evaluation objectives, evidence layers, and claim boundaries used in the manuscript

Question	Evidence used	Boundary
Can ML-KEM be added without an extra message?	KE1–KE2–KE3 syntax, wire-size accounting, and serializer tests	This is an OPAQUE-like migration profile, not a claim of full RFC 9807 conformance.
Is the post-quantum material transcript-bound?	Account context, extended transcript equations, MAC confirmation, and mutation tests	The combiner argument is scoped to the stated extractor interpretation.
What formal evidence supports the design?	Split ProVerif models and a surrogate Tamarin 4DH model	Symbolic evidence does not machine-check the Rust implementation or side-channel behavior.
What is the measured cost?	Criterion benchmarks on the stated Linux/Xeon platform	Timings are platform-specific and require recalibration on mobile, browser, and embedded targets.

3. **A reproducible cost and trust-boundary analysis.** The artifact reports message expansion, Criterion benchmarks, and operational assumptions for $\sigma_{\text{opr}}f$, showing that the dominant latency comes from Argon2id while the main new overhead is wire-format expansion.

The remainder of the paper follows a claim-evidence structure. Section 2 positions the construction against OPAQUE, post-quantum PAKE, and hybrid-composition work. Section 3 states the objective, roles, assets, and threat boundary. Section 4 specifies the protocol syntax and key schedule. Section 5 develops the reduction sketches, the attack analysis, and the symbolic security interpretation, and is explicit about their limits. Section 6 reports measurements and then separates implementation diagnostics from the main results. Section 7 discusses what the combined evidence does and does not establish, and Section 8 concludes.

1.4. Terminology

Table 2 fixes the small vocabulary needed before the protocol is defined. Two entries carry weight later. The distinction between payload and wire format, and the role of `acct` as cryptographic context, recur throughout the construction.

The formal exposition uses `acct` for the account identifier and $\sigma_{\text{opr}}f$ for the root secret from which server-side OPRF keys are derived. Literal ASCII domain-separation strings are replaced below with the symbols $L_{\text{opr}}f$, L_{seed} , L_{key} , L_{ksf} , L_{salt} , L_{ctx} , L_{τ} , and L_{comb} . For the purposes of formal analysis, what matters is their stability and mutual distinctness, not the concrete textual

Table 2: Terminology used in the paper

Term	Meaning in this paper
Wire format	The complete byte-level representation of a message on the channel, including the version prefix and protocol service fields
Payload	The message body excluding the version prefix and any external wrappers
Account identifier (acct)	A stable byte string identifying the user’s account. In this construction it is cryptographic context, not only a database lookup key
Initiator $\mathcal{C}$	The party that holds the password, creates KE1, verifies KE2, and sends KE3
Record-holding responder $\mathcal{S}$	The augmented-PAKE party that stores the registration record, evaluates the OPRF, creates KE2, and verifies KE3
Augmented PAKE (aPAKE)	A class of password-authenticated key-exchange protocols in which the server stores a verifier rather than the password itself or its direct equivalent
Key-derivation scheme	The sequence of extraction and expansion steps that produces session and auxiliary keys
Research artifact	Source code, formal models, archived verification logs, tests, benchmark harnesses, and adapter documentation needed for independent reproduction of the reported results

representation of the implementation string. For byte strings, $\text{Trunc}_n(X)$ denotes the leftmost n octets of X . Advantage expressions in the manuscript are scoped labels for the argument being made, not theorem-level security definitions for an independent PAKE family. When a bound is written below, it should be read under the stated compromise model, primitive assumptions, and modeling abstractions of that subsection.

2. Related Work and Existing Solutions

Because post-quantum standardization is still in motion, we are careful about the status of each source we cite. Final standards and RFCs such as FIPS 203 and RFC 9807 serve as normative anchors. Peer-reviewed papers carry the established PAKE and post-quantum results. ePrints and Internet-Drafts supply current evidence that we treat as provisional. Technical white papers enter only for contextual calibration.

2.1. Password-Authenticated Key-Exchange Protocols

Password-authenticated key exchange (PAKE) names a class of protocols that let two parties who share only a low-entropy secret, a password, agree on an authenticated session key that resists offline dictionary attacks [21]. The literature sorts these protocols into two broad categories:

- *Balanced PAKE*, in which both parties store equivalent password-derived representations. Representative examples are EKE [22], SPAKE2 [23], and CPace [25].
- *Augmented PAKE (aPAKE)*, in which the server stores only a verifier rather than the password or its direct equivalent. Representative examples are SRP [26], AuCPace [25], and OPAQUE [4].

Table 3: Comparative characteristics of selected PAKE protocols

Property	EKE	SRP	SPAKE2	OPAQUE
Type	Balanced	Augmented	Balanced	Augmented
Non-equivalent server record	No	Partial	No	Yes
Pre-computation resistance	No	No	No	Yes
Forward secrecy	Yes	Yes	Yes	Yes
Resistance to server-record compromise	No	Partial	No	In the passive-database model
Formal security proof	Ideal-cipher model	None	ROM	UC
Quantum resistance	No	No	No	No

The OPAQUE entry in Table 3 on server-record compromise should be read in the narrow but standard sense, namely compromise of the authentication-record database alone. It says nothing about the loss of a global server secret from which the OPRF keys may be derived.

2.2. OPAQUE: A Formal Overview

OPAQUE [4] comprises a registration phase and an authentication phase, both built around an oblivious pseudorandom function (OPRF). Standard IRTF OPAQUE [5] uses triple Diffie–Hellman (3DH): $dh_1 = sk_S \cdot PK_C$ (mutual authentication), $dh_2 = sk_S \cdot E_C$, $dh_3 = e_S \cdot PK_C$ (forward secrecy). The construction analyzed here adds a fourth Diffie–Hellman term, $dh_4 = e_S \cdot E_C$, which binds the two ephemeral components within a single transcript. At the level of 4DH intuition, this term is meant to complicate unknown key-share

(UKS)-like substitutions, and we treat it as a design rationale rather than an independently proved property. The same term carries the forward-secrecy intuition. Once both long-term keys sk_C and sk_S leak, the static-secret terms dh_1 , dh_2 , and dh_3 become reconstructable from the public transcript, whereas $dh_4 = e_S \cdot E_C = e_C \cdot E_S$ binds the two session ephemerals and stays out of reach unless an ephemeral scalar is recovered. We tie this reading to the scoped forward-secrecy sketch in Eq. (21). Throughout this paper “4DH” denotes these four Diffie–Hellman terms in the key schedule, not a fourth authentication message, and the message flow stays KE1–KE2–KE3. Baseline OPAQUE has been proved secure in the Universal Composability (UC) model [4, 27].

Oblivious pseudorandom function (OPRF). The OPRF core uses a hash-to-group map into Ristretto255 [28], following the prime-order-group OPRF specified in RFC 9497 [24]:

$$\begin{aligned}\alpha &= r \cdot H'(\text{ctx} \parallel x), \\ \beta &= k \cdot \alpha, \\ F_k(x) &= r^{-1} \cdot \beta,\end{aligned}\tag{1}$$

where $r \xleftarrow{\$} \mathbb{Z}_q^*$ is a random initiator-side blinding scalar, k is the responder’s OPRF key, and ctx is the domain-separation context for the invocation.

2.3. Post-Quantum Cryptography and the NIST 2024 Standards

ML-KEM-768 is selected at NIST security level 3 (roughly aligned with AES-192). Its parameters are $|pk| = 1184$ bytes, $|ct| = 1088$ bytes, and $|ss| = 32$ bytes under IND-CCA2 security. The abstraction is

$$\text{KeyGen}() \rightarrow (pk, sk); \quad \text{Encaps}(pk) \rightarrow (ct, ss); \quad \text{Decaps}(sk, ct) \rightarrow ss. \tag{2}$$

2.4. Rationale for Selecting ML-KEM-768

Our choice of ML-KEM-768 follows from the deployment constraints of interactive password authentication. ML-KEM-512 sends smaller messages at lower overhead, but it only reaches NIST level 1. ML-KEM-1024 raises the nominal security level at the cost of larger messages and more transport. Because the protocol already runs a comparatively expensive key-stretching function (KSF), the criterion that matters here is the balance among cryptographic strength, message size, and deployability.

ML-KEM-768 sits at a middle operating point in that trade-off space. Within the ML-KEM family it matches NIST level 3 to the long-term protection of account credentials and session secrets, and it avoids the heavier transport burden of ML-KEM-1024. The values pk_{kem} and ct_{kem} do enlarge messages relative to classical OPAQUE, yet they leave the protocol usable in an interactive setting.

2.5. Design Alternatives Considered

Several hybridization strategies are available, and each carries its own operational trade-offs. A purely transport-layer migration can adopt a post-quantum or hybrid TLS key exchange and leave the application authentication protocol untouched. That path eases deployment, but it never makes the password-authentication transcript itself hybrid. Replacing the OPRF with a post-quantum OPRF would attack the password-randomization layer more directly, though efficient and standardized post-quantum OPRFs remain less mature than ML-KEM-based key establishment. A third route stores long-lived post-quantum material in the registration record, which raises the migration pressure on account state and enlarges the server-side record that operators must protect. A fourth route keeps the classical PAKE and a post-quantum KEM as separate handshakes and composes their outputs, which leaves the PAKE internals untouched but needs a separate composition argument so that two independent exchanges are not read as one authenticated transcript.

The construction studied here takes a narrower route. It keeps the OPRF and envelope structure close to OPAQUE, adds ML-KEM-768 only in the ephemeral authentication path, and binds the resulting public KEM values into the same transcript that authenticates the classical exchange. This choice does not solve every post-quantum PAKE problem, but it yields a concrete migration point whose wire format, implementation cost, and trust boundary can all be measured. Table 4 lists the alternatives we considered and the reason we settled on the ephemeral ML-KEM path.

2.6. Recent Approaches to Post-Quantum PAKE (2023–2026)

Between 2023 and 2026, several research lines became relevant to post-quantum PAKE and hybrid aPAKE design.

KEM-based schemes show that compact lattice-based PAKE is feasible. CHIC [17] is a representative example, pairing a compact construction with a UC-security argument and an implementation-level evaluation. Alongside

Table 4: Hybridization strategies and their implications for this work

Strategy	Benefit	Limitation for the present objective
Hybrid transport only	Minimal application changes when the surrounding channel already supports post-quantum key exchange	The PAKE transcript and application-level authentication state remain classical.
Post-quantum OPRF replacement	Targets the password-randomization layer directly	Efficient post-quantum OPRF choices are less standardized and would change the registration and envelope design more deeply.
Persistent PQ material in the record	Can attach post-quantum state to the account itself	Enlarges long-lived account records and complicates staged migration.
Separate PAKE and KEM composition	Keeps the classical PAKE largely unchanged	Requires a separate composition argument and careful binding to avoid treating two independent handshakes as one authenticated exchange.
Ephemeral ML-KEM in the OPAQUE transcript	Preserves three messages, keeps PQ material out of the registration record, and exposes concrete wire-format costs	Leaves the OPRF layer classical and requires a carefully scoped hybrid-combiner interpretation.

it, UC- and QROM-oriented results such as [18] tighten the compositional guarantees available in the post-quantum setting. This progress matters, yet none of it directly resolves the migration problem for deployed OPAQUE-like systems, where asymmetric initiator–responder roles, wire-format stability, and compatibility with existing authentication workflows take center stage.

A second line of work looks at replacing classical OPRF components and at compiling KEM primitives into PAKE. Lattice-based post-quantum OPRF constructions stay more communication-intensive than their elliptic-curve counterparts [29], and generalized KEM-to-PAKE approaches demand careful handling of ideal-cipher assumptions, hashing into public-key spaces, and low-entropy secrets [17]. Isogeny-based approaches, by contrast, lost most of their practical relevance once the Castryck–Decru attacks broke SIDH-derived constructions [30].

Recent work also frames hybridization itself as a distinct design problem. PAKE combiners [19] and UC analyses of hybrid PAKE [20] mostly target proof structure and composition, and the CFRG-related hybrid PAKE draft [31] signals active standardization interest. Our contribution is narrower than any of these. It is neither a new UC-secure PAKE family nor a general combiner theorem. What it offers is an OPAQUE migration profile with explicit transcript syntax, no extra authentication message, no persistent post-quantum account state, a stated σ_{oprf} trust boundary, and reproducible implementation measurements.

Set against CHIC and the UC/QROM-oriented post-quantum PAKEs, our construction puts migration from an augmented OPAQUE setting ahead of a clean-slate PAKE design. Set against PAKE-combiner work, it supplies concrete wire syntax and artifact evidence instead of a general composition proof. Set against transport-only hybridization, it folds the post-quantum contribution into the password-authentication transcript itself.

3. Problem Statement

3.1. Problem Formulation and Threat Model

Let $\mathcal{C}$ be the *initiator* and $\mathcal{S}$ be the *record-holding responder*. The initiator holds the password pwd and completes OPRF finalization locally. The responder stores the registration record rec and derives account-specific OPRF keys from σ_{oprf} . We use this vocabulary throughout the threat model, the figures, the symbolic models, and the implementation discussion, and we treat the deployment terms client and server only as explanatory aliases. Let $\mathbb{G}$ denote the prime-order Ristretto255 group of order q with generator g .

Assets to be protected:

- the password pwd ; the static keys sk_C, sk_S ; the ephemeral keys $e_C, e_S, sk_{\text{kem}}$;
- the registration record $\text{rec} = (\text{Envelope}, PK_C)$;
- the session key K_s , the master key K_m , and the shared secret ss_{kem} .

Adversarial model. Three adversarial classes are considered:

- *Classical adversary* $\mathcal{A}_C$: a probabilistic polynomial-time active man-in-the-middle capable of compromising server-side storage;
- *Quantum adversary* $\mathcal{A}_Q$: an attacker modeled as capable of solving DLP/ECDLP via Shor’s algorithm and pursuing a “harvest-now-decrypt-later” strategy;
- *Composite adversary* $\mathcal{A}_{CQ}$: an active man-in-the-middle equipped with a quantum computational oracle.

Threat-model boundary. The adversary controls the network and may record, replay, delay, drop, and modify protocol messages. The database-compromise scenario gives the adversary the registration record and associated public metadata, but not the root secret $\sigma_{\text{opr}}f$ from which account-specific OPRF keys are derived. The model also permits post-session compromise of long-term static protocol keys when discussing forward secrecy. It does not include full endpoint compromise during an active session, malicious random number generation, password-reset workflows, resource-exhaustion denial-of-service analysis, or microarchitectural side channels. These excluded cases are operationally important, but they require different mechanisms from the protocol-level construction studied here.

Operationally, the database-only claim requires $\sigma_{\text{opr}}f$ to be managed outside the password-record store and outside ordinary database backup, analytics, and logging paths. A conforming deployment should keep $\sigma_{\text{opr}}f$ inside a KMS, HSM, or equivalent secret-management boundary and grant derivation access only to the authentication service. Plaintext $\sigma_{\text{opr}}f$ should stay out of snapshots, crash dumps, and support bundles, and every derivation call should be audited. The deployment also needs an incident-response path that treats suspected $\sigma_{\text{opr}}f$ exposure as verifier compromise, which then requires root rotation, account re-enrollment or password reset, and temporary rate tightening. Table 5 makes this boundary explicit for the main claims, and Figure 1 shows the same boundary as a map.

Protocol requirements:

- R1** (OPAQUE-derived goals): password secrecy, forward secrecy, mutual authentication, and resistance to offline guessing;
- R2** (Post-quantum reinforcement): inclusion of a KEM-based contribution against “harvest-now-decrypt-later” exposure;
- R3** (Hybrid-source interpretation): the key-material combiner should support a stated extractor-based reading in the adopted model, so that the paper can identify precisely which source assumptions remain necessary for final-key recovery;
- R4** (Interaction preservation): preservation of three protocol messages and 1.5 network round trips;
- R5** (Implementability): a reproducible reference implementation in Rust suitable for further analysis and engineering.

Table 5: Threat-model boundary used for the main security claims

Scenario	Included in the claim	Boundary
Active network attack	Full Dolev–Yao control of messages and transcript manipulation	Does not model traffic analysis or availability attacks.
Database disclosure	Registration record, stored public keys, and protocol metadata may be exposed	Offline resistance assumes σ_{prf} remains outside the compromised store.
Post-session key exposure	Long-term static keys may be compromised after completion	Session-local ephemerals and ML-KEM decapsulation state are assumed destroyed.
Quantum attack on classical DH	Classical DH terms may be treated as recoverable in the long-term migration argument	The post-quantum layer is then relied upon under the ML-KEM assumption.
Implementation behavior	Serialization, state lifecycle, version handling, and external-adaptor errors are checked by tests	Tests do not establish the absence of side channels or memory-safety bugs in external crates.

3.2. Formal Definitions

Definition 3.1 (KEM). (KeyGen, Encaps, Decaps): $\text{KeyGen}() \rightarrow (pk, sk)$, $\text{Encaps}(pk) \rightarrow (ct, ss)$, and $\text{Decaps}(sk, ct) \rightarrow ss$. *IND-CCA2 security means that no polynomial-time adversary equipped with a decapsulation oracle can distinguish the real ss from a random string.*

Definition 3.2 (Hybrid aPAKE in the sense of this paper). *Protocol Π between $\mathcal{C}(\text{pwd})$ and $\mathcal{S}(\text{rec})$, in which*

$$\begin{aligned} \text{IKM}_{\text{cl}} &= \text{Encode}(dh_1) \parallel \text{Encode}(dh_2) \\ &\quad \parallel \text{Encode}(dh_3) \parallel \text{Encode}(dh_4), \\ \text{IKM}_{\text{pq}} &= ss_{\text{kem}}, \end{aligned}$$

and the final pre-key is formed as

$$\text{PRK} = \text{HKDF-Extract}(\text{salt}_\tau, \text{IKM}_{\text{cl}} \parallel \text{IKM}_{\text{pq}}).$$

The subsequent analysis examines the extent to which such a construction is compatible with the hybrid-source interpretation used within the threat model considered here.

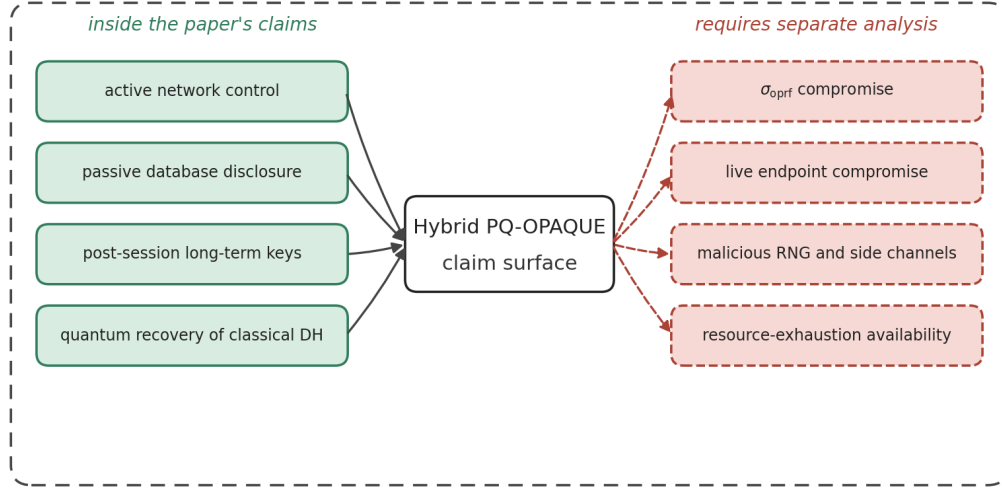


Figure 1: Threat-boundary map for the main claims. The construction is analyzed against active network control, database disclosure, post-session long-term key exposure, and later quantum recovery of classical DH terms. Compromise of the OPRF root secret, live endpoint compromise, malicious randomness, side channels, and availability attacks are outside the claim surface.

Definition 3.3 (Pseudorandomness of HKDF-Extract). *HKDF-Extract based on HMAC-SHA-512 is modeled as an extractor: if IKM contains hidden computational entropy relative to the adversary, then the output PRK is computationally indistinguishable from uniform up to advantage at most ε .*

Notation: $\mathbb{G}$ denotes Ristretto255 of order $|\mathbb{G}| = q$. Unless stated otherwise, lowercase letters denote scalars and uppercase letters denote curve points. The symbol $\parallel$ denotes concatenation, $H(\cdot)$ denotes SHA-512, and $\mathcal{K} = (K_s, K_m)$ denotes the session and master keys. The map $H'(\cdot)$ hashes into $\mathbb{G}$, $\text{Encode}(\cdot)$ is the canonical 32-byte encoding of a group element, and $\text{ScalarReduce}(\cdot)$ reduces a byte string to a scalar in $\mathbb{Z}_q$.

4. The Proposed Hybrid PQ-OPAQUE Protocol

4.1. Architecture and Overview

Hybrid PQ-OPAQUE extends OPAQUE [5] into the hybrid post-quantum setting while keeping the augmented-PAKE asymmetry intact. In an honest run the password is never sent to the record-holding responder and never reconstructed by it. Authentication stays a three-message KE1–KE2–KE3

exchange, and the only post-quantum state it adds is ephemeral ML-KEM-768 material, namely pk_{kem} in KE1 and ct_{kem} in KE2.

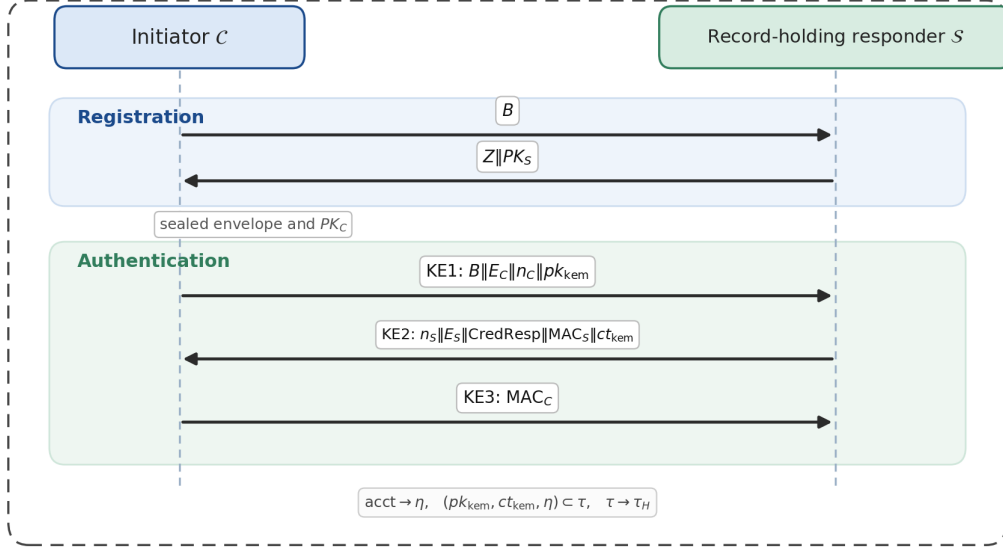


Figure 2: Hybrid PQ-OPAQUE message flow between initiator $\mathcal{C}$ and record-holding responder $\mathcal{S}$. ML-KEM public and ciphertext material are carried in KE1 and KE2, while account and transcript binding feed the authenticated key schedule without adding an authentication message.

Table 6: Cryptographic primitives used in Hybrid PQ-OPAQUE

Component	Primitive	Implementation	Security level
OPRF group	Ristretto255	curve25519-dalek	128 bits
Hash function	SHA-512	sha2	256 bits
KDF (extract)	HKDF-Extract (HMAC-SHA-512)	hmac + sha2	256 bits
KDF (expand)	HKDF-Expand (HMAC-SHA-512)	hmac + sha2	256 bits
MAC	HMAC-SHA-512	hmac + sha2	256 bits
KSF	Argon2id (MODERATE)	argon2	Adaptive
AE	XSalsa20-Poly1305	crypto_secretbox	256 bits
KEM	ML-KEM-768 (FIPS 203)	ml-kem	NIST Level 3
Classical exchange	4DH (Ristretto255)	curve25519-dalek	128 bits
Constant-time comparison	Bytewise without early exit	subtle	—

Figure 2 shows where these values, together with the account-context hash η , enter the transcript and key schedule.

Table 6 fixes the concrete primitive choices used by the construction and by the implementation evaluated below.

The authentication-message sizes in Table 7 report payload size separately from wire-format size, where the wire-format figure also counts the one-byte protocol version prefix.

Table 7: Authentication-message sizes in payload and wire format

Message	Classical payload	Hybrid payload	Hybrid wire format	Post-quantum overhead
KE1	88 bytes	1272 bytes	1273 bytes	+1184 (pk_{kem})
KE2	288 bytes	1376 bytes	1377 bytes	+1088 (ct_{kem})
KE3	64 bytes	64 bytes	65 bytes	0
Total	440 bytes	2712 bytes	2715 bytes	+2272 (+516%)

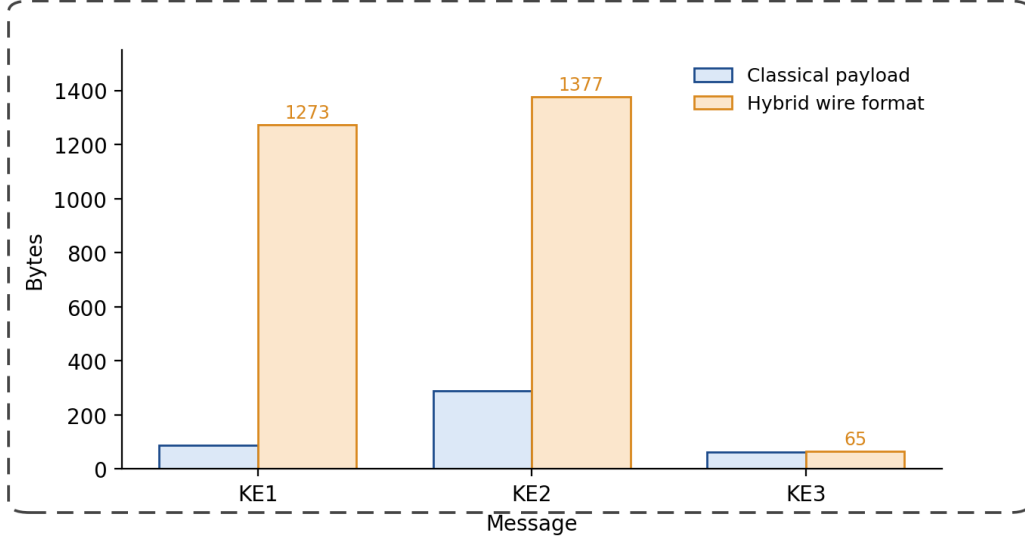


Figure 3: Comparison of authentication-message sizes: the additional wire-format overhead is concentrated in KE1 and KE2, whereas KE3 remains essentially unchanged.

Figure 3 shows that the post-quantum layer leaves finalization almost untouched and concentrates the added transport cost in the first two messages, the ones that carry pk_{kem} and ct_{kem} .

4.2. Normative Wire Format and Rejection Rules

Every authentication message is a fixed-width byte string. Its first byte is the protocol version $v = 0x01$. KE1, KE2, and KE3 carry no endian-sensitive integer fields, so all components listed below are copied verbatim as octet strings at fixed offsets. The account identifier `acct` enters as external authenticated-context input rather than as a field inside these messages.

Parsing is fail-closed. A message must have the exact expected length before any field is interpreted. An unsupported version is reported separately from a malformed length at the public API boundary, whereas MAC, envelope, and password failures all collapse into a single authentication failure that limits account and stage fingerprinting. Ristretto encodings must decompress canonically, and ML-KEM public keys and ciphertexts must match the FIPS 203 lengths shown in Table 8. Malformed public inputs are rejected outright rather than silently normalized.

Table 8: Fixed authentication-message layout

Message	Offset range	Field	Length and validation rule
KE1	0	v	1 byte, must equal 0x01
KE1	1–32	B_{auth}	32-byte blinded OPRF group element, canonical Ristretto encoding required
KE1	33–64	E_C	32-byte initiator ephemeral Ristretto public key
KE1	65–88	n_C	24-byte initiator nonce
KE1	89–1272	pk_{kem}	1184-byte ML-KEM-768 public key
KE2	0	v	1 byte, must equal 0x01
KE2	1–24	n_S	24-byte responder nonce
KE2	25–56	E_S	32-byte responder ephemeral Ristretto public key
KE2	57–224	CredResp	168-byte credential response
KE2	225–288	MAC_S	64-byte HMAC-SHA-512 confirmation token
KE2	289–1376	ct_{kem}	1088-byte ML-KEM-768 ciphertext
KE3	0	v	1 byte, must equal 0x01
KE3	1–64	MAC_C	64-byte HMAC-SHA-512 confirmation token

4.3. Registration Phase

Step 1 ($\mathcal{C} \rightarrow \mathcal{S}$). Generation of the static pair (sk_C, PK_C) followed by OPRF blinding:

$$r \xleftarrow{\$} \mathbb{Z}_q^*, \quad B = r \cdot H'(L_{\text{oprf}} \| 0x00 \| \text{pwd}). \quad (3)$$

RegistrationRequest = B (32 bytes).

Step 2 ($\mathcal{S} \rightarrow \mathcal{C}$). Derivation of the OPRF key:

$$s_{\text{oprf}} = \text{Trunc}_{32}(\text{HMAC}(\sigma_{\text{oprf}}, L_{\text{seed}})), \quad (4)$$

$$k_{\text{oprf}} = \text{ScalarReduce}(\text{HMAC}(s_{\text{oprf}}, L_{\text{key}} \| \text{acct} \| \text{ctr})), \quad (5)$$

where ctr is the smallest one-byte counter for which the resulting scalar is non-zero. The derivation is two-level on purpose. The seed s_{oprf} is a domain-separated value computed once from σ_{oprf} , and each account key is derived from the seed rather than from the root directly, which keeps every account key two HMAC steps removed from the root secret. The responder computes $Z = k_{\text{oprf}} \cdot B$. The key k_{oprf} is then immediately destroyed. The response is $Z \| PK_S$ (64 bytes).

Step 3 ($\mathcal{C}$). OPRF unblinding:

$$U = r^{-1} \cdot Z.$$

OPRF output:

$$y_{\text{oprf}} = H(L_{\text{oprf}} \| 0\mathbf{x}01 \| \text{pwd} \| U). \quad (6)$$

KSF input and salt:

$$x_{\text{ksf}} = H(L_{\text{ksf}} \| y_{\text{oprf}} \| \text{pwd}), \quad (7)$$

$$\text{salt}_{\text{ksf}} = \text{Trunc}_{16}(H(L_{\text{salt}} \| y_{\text{oprf}})). \quad (8)$$

The randomized password is then computed with Argon2id [32] as

$$\text{rwd} = \text{Argon2id}(x_{\text{ksf}}, \text{salt}_{\text{ksf}}; m = 256 \text{ MiB}, t = 3, p = 1). \quad (9)$$

The MODERATE profile is the reference benchmark profile, not a universal protocol constant. It targets server-class, desktop, and business-to-business (B2B) deployments where approximately 256 MiB of memory per authentication attempt is acceptable. Constrained clients should recalibrate the KSF profile and must not inherit the statement that ML-KEM overhead is practically negligible without remeasurement. Table 9 lists the KSF profiles and indicates which one is measured here.

Table 9: KSF profiles and benchmark status

Profile	Parameters	Measured here?	Intended use
LIGHT	$m = 64 \text{ MiB}, t = 2, p = 1$	No	Constrained or latency-sensitive clients requiring separate calibration
MODERATE	$m = 256 \text{ MiB}, t = 3, p = 1$	Yes	Reference server-class and desktop/B2B profile used for the reported measurements
HARD	$m \geq 512 \text{ MiB}, t \geq 3, p = 1$	No	High-value accounts or environments where memory cost is acceptable

The envelope seals $PK_S \| sk_C \| PK_C$ under XSalsa20-Poly1305 with a key derived from rwd through HKDF, giving $\text{RegistrationRecord} = \text{Envelope} \| PK_C$ (168 bytes of payload and 169 bytes in wire format, including the version prefix).

4.4. Binding of Account Identifier and Context

We pin down context that most abstract descriptions of OPAQUE leave implicit. Every wire-format message begins with a one-byte version prefix $v = 0x01$, which separates the payload from the full wire format and lets the parser reject unsupported versions. The protocol also names the account explicitly through an identifier `acct`, a stable byte string that we treat as cryptographic context rather than as a mere database lookup key.

Define the account-context hash used by the evaluated artifact as a domain-separated binding of the account identifier:

$$\eta = H(L_{\text{ctx}} \parallel \text{acct}). \quad (10)$$

Here `acct` arrives as a separate API input, is rejected when empty, and is never serialized inside KE1, KE2, or KE3. In the current artifact the version byte v is validated at the wire-format layer rather than folded into η . A future interoperability profile could make this binding length-prefixed and version-indexed, but that would be a new wire-format profile decision and not a claim about the implementation we evaluate here. Distinct domain-separation labels keep the baseline OPRF, KSF, and transcript contexts apart from the added account-binding and hybrid-combiner contexts. The version indices carried in those labels are local domain identifiers, not global version numbers for the scheme as a whole. The identifier `acct` enters the protocol at two points:

1. through derivation of an account-specific OPRF key, as in Eq. (5);
2. through inclusion of η , defined in Eq. (10), in the extended transcript (17), and hence in the hybrid combiner (18) and the derived keys (19).

This binding makes cross-account transplantation visible in transcript verification even when the password is identical, and it makes the role of account identity explicit in the cryptographic specification.

4.5. Authentication Phase and Hybrid Key Exchange

4.5.1. Generation of KE1 from Initiator to Record-Holding Responder

The initiator first samples the OPAQUE authentication randomness and creates the ephemeral ML-KEM key pair advertised in the first message:

$$\begin{aligned} r_{\text{auth}} &\stackrel{\$}{\leftarrow} \mathbb{Z}_q^*, & B_{\text{auth}} &= r_{\text{auth}} \cdot H'(L_{\text{oprf}} \parallel 0x00 \parallel \text{pwd}), \\ e_C &\stackrel{\$}{\leftarrow} \mathbb{Z}_q^*, & E_C &= e_C \cdot g, \\ n_C &\stackrel{\$}{\leftarrow} \{0, 1\}^{192}, & (pk_{\text{kem}}, sk_{\text{kem}}) &\leftarrow \text{ML-KEM-768.KeyGen}(). \end{aligned} \quad (11)$$

The resulting first authentication message is

$$\begin{aligned} \text{KE1} &= B_{\text{auth}} \parallel E_C \parallel n_C \parallel pk_{\text{kem}}, \\ |\text{KE1}| &= 32 + 32 + 24 + 1184 = 1272 \text{ bytes.} \end{aligned} \quad (12)$$

With the one-byte protocol version prefix, the wire message $v \parallel \text{KE1}$ occupies 1273 bytes. The value B_{auth} is a fresh authentication-phase OPRF blinding value and is independent of the registration-phase value B in Eq. (3). In the implementation API and wire parser, this same 32-byte field is named `credential_request`.

4.5.2. Generation of KE2 from Record-Holding Responder to Initiator

Responder ephemeral material:

$$e_S \xleftarrow{\$} \mathbb{Z}_q^*, \quad E_S = e_S \cdot g, \quad n_S \xleftarrow{\$} \{0, 1\}^{192}. \quad (13)$$

Authentication OPRF evaluation: The responder re-derives the account OPRF key k_{oprf} from σ_{oprf} and `acct` as in Eq. (5), rebuilding it per request because the key was destroyed after registration, and evaluates it on the initiator's blinded element B_{auth} from KE1:

$$Z_{\text{auth}} = k_{\text{oprf}} \cdot B_{\text{auth}}. \quad (14)$$

The value Z_{auth} forms the leading 32 bytes of `CredResp`, after which k_{oprf} is destroyed.

Four-way Diffie–Hellman (4DH):

$$dh_1 = sk_S \cdot PK_C, \quad dh_2 = sk_S \cdot E_C, \quad dh_3 = e_S \cdot PK_C, \quad dh_4 = e_S \cdot E_C. \quad (15)$$

ML-KEM-768 encapsulation:

$$(ct_{\text{kem}}, ss_{\text{kem}}) \leftarrow \text{ML-KEM-768.Encaps}(pk_{\text{kem}}). \quad (16)$$

Extended transcript (including post-quantum elements):

$$\begin{aligned} \tau &= E_C \parallel E_S \parallel n_C \parallel n_S \parallel PK_C \parallel PK_S \parallel \text{CredResp} \parallel pk_{\text{kem}} \parallel ct_{\text{kem}} \parallel \eta, \\ \tau_H &= H(L_\tau \parallel \tau). \end{aligned} \quad (17)$$

Hybrid combiner:

$$\begin{aligned}
\text{IKM}_{\text{cl}} &= \text{Encode}(dh_1) \parallel \text{Encode}(dh_2) \parallel \text{Encode}(dh_3) \parallel \text{Encode}(dh_4), \\
\text{IKM}_{\text{pq}} &= ss_{\text{kem}}, \\
\text{IKM}_{\text{hyb}} &= \text{IKM}_{\text{cl}} \parallel \text{IKM}_{\text{pq}} \quad (128 + 32 = 160 \text{ bytes}), \\
\text{salt}_{\tau} &= L_{\text{comb}} \parallel \tau_H, \\
\text{PRK} &= \text{HKDF-Extract}(\text{salt}_{\tau}, \text{IKM}_{\text{hyb}}).
\end{aligned} \tag{18}$$

Figure 4 shows the resulting dataflow from the classical 4DH terms, the ML-KEM shared secret, and the transcript hash into the derived session keys.

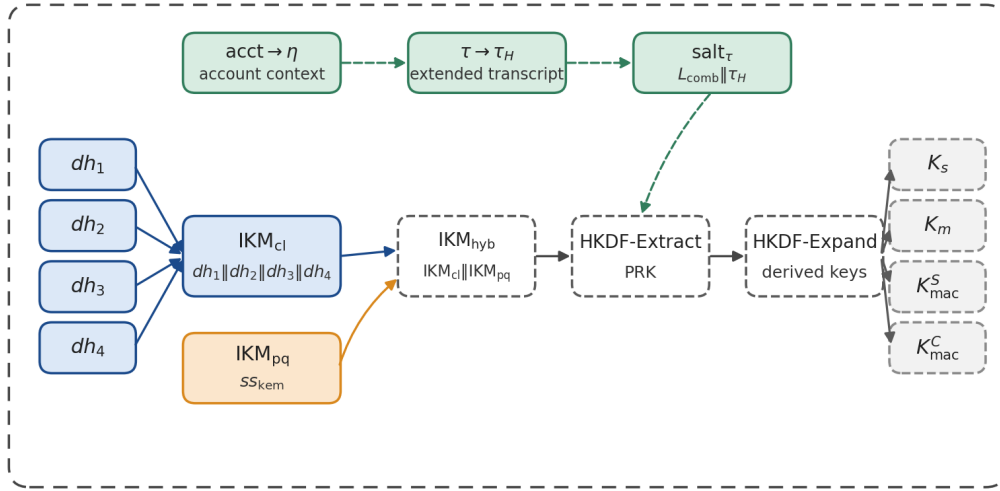


Figure 4: Transcript-bound hybrid key schedule. Classical 4DH material and the ML-KEM shared secret are extracted under a transcript-derived salt before expansion into session, master, and confirmation keys.

Key derivation. Let Λ_s , Λ_m , Λ_{macS} , and Λ_{macC} denote fixed labels for the session key, the master key, and the two confirmation keys:

$$\begin{aligned}
K_s &= \text{HKDF-Expand}(\text{PRK}, \Lambda_s, 64), \\
K_m &= \text{HKDF-Expand}(\text{PRK}, \Lambda_m, 32), \\
K_{\text{mac}}^S &= \text{HKDF-Expand}(\text{PRK}, \Lambda_{\text{macS}}, 64), \\
K_{\text{mac}}^C &= \text{HKDF-Expand}(\text{PRK}, \Lambda_{\text{macC}}, 64).
\end{aligned} \tag{19}$$

The resulting second authentication message is

$$\begin{aligned} \text{KE2} = n_S \| E_S \| \text{CredResp} \| \\ \text{HMAC}(K_{\text{mac}}^S, \tau) \| ct_{\text{kem}} \quad (1376 \text{ bytes of payload}). \end{aligned} \quad (20)$$

In wire format, the message is $v \| \text{KE2}$ and occupies 1377 bytes. The credential response itself decomposes as $\text{CredResp} = Z_{\text{auth}} \| \text{Envelope}$, the 32-byte evaluated OPRF element followed by the 136-byte envelope ($32 + 136 = 168$ bytes).

4.5.3. Generation of KE3 from Initiator to Record-Holding Responder

1. OPRF finalization, opening of the envelope, and recovery of the responder public key, initiator static key, and initiator public key;
2. Mirror 4DH: $dh_1 = sk_C \cdot PK_S$, $dh_2 = e_C \cdot PK_S$, $dh_3 = sk_C \cdot E_S$, $dh_4 = e_C \cdot E_S$ (additional binding of the ephemeral components at the level of 4DH intuition);
3. Decapsulation: $ss_{\text{kem}} = \text{ML-KEM-768.Decaps}(sk_{\text{kem}}, ct_{\text{kem}})$; sk_{kem} is destroyed immediately;
4. Re-derivation of the same PRK; verification of $\text{HMAC}(K_{\text{mac}}^S, \tau)$ by means of a constant-time bitwise comparison without early exit;
5. $\text{KE3} = \text{HMAC}(K_{\text{mac}}^C, \tau)$ (64 bytes of payload, 65 bytes including the version prefix).

4.5.4. Completion at the Record-Holding Responder

The responder verifies KE3 with a constant-time comparison routine. On success the pair (K_s, K_m) becomes available to both parties, which confirms mutual authentication. Completion is fail-closed. If KE3 is malformed, expired, or fails MAC verification after KE2 has been generated, no session key is exported, the responder-held copies of K_s , K_m , ss_{kem} , the expected initiator MAC, and the responder ephemeral secrets are zeroized, and the state moves to a terminal or invalidated condition so the same KE2 state cannot be retried as an authentication oracle.

4.6. Transcript-Binding Check

The transcript definition in Eq. (17) is the normative binding point of the hybrid construction. It forces both pk_{kem} and ct_{kem} to be MAC-authenticated alongside the listed classical authentication fields and the account-context hash η . The authentication OPRF blinding value B_{auth} is validated and

evaluated on the OPRF path instead of being placed in τ , and the version byte is handled by the wire-format parser before the transcript is built. An implementation that drops any of the listed post-quantum fields or omits η from τ is implementing a different protocol. What remains binds the classical and post-quantum key-schedule inputs to a concrete account under the versioned wire format in which the messages are parsed.

5. Security Analysis

5.1. Formal Analysis of Security Properties

The arguments below address the adversary classes introduced in Section 3.1. Mutual authentication and MAC confirmation are stated against the classical active attacker $\mathcal{A}_C$, post-quantum confidentiality of the session key against the harvest-now-decrypt-later attacker $\mathcal{A}_Q$, and the combined guarantee against the composite attacker $\mathcal{A}_{CQ}$.

Offline guessing under a fixed trust boundary. We keep the compromise case narrow on purpose. The adversary may read the passive server-side storage rec and the observable transcripts, but never σ_{oprf} . Under that condition the record gives no efficient offline verifier for dictionary candidates except through a break of OPRF pseudorandomness or envelope security. Once σ_{oprf} leaks, the situation inverts. The attacker can then evaluate the account-specific OPRF path, and what remains is a dictionary search whose per-guess cost is dominated by Argon2id. The claim thus speaks to one specific operational boundary, not to every conceivable server compromise.

Proof sketch. Let S_0, S_1, S_2 denote the real database-disclosure experiment, the experiment with randomized OPRF output, and the experiment with a hidden envelope plaintext, respectively. Let $\mathcal{Q}_{\text{on}}$ be the set of password candidates submitted through online protocol attempts. **Case 1:** σ_{oprf} not compromised. Without the account-specific OPRF key, the adversary cannot compute y_{oprf} via (6), and hence cannot reconstruct x_{kfs} , salt_{kfs} , or rwd via (9). The envelope record therefore does not provide an effective offline password-checking oracle. Replacing the OPRF output with a random value yields

$$|\Pr[S_1] - \Pr[S_0]| \leq \text{Adv}_{\text{OPRF}}^{\text{PRF}} \leq \text{Adv}_{\text{CDH}}^{\text{Ristretto255}}.$$

Envelope-hiding transition. Next replace the envelope with an encryption of a random string, reducing the change in success probability to the

authenticated-encryption security of XSalsa20-Poly1305:

$$|\Pr[S_2] - \Pr[S_1]| \leq \text{Adv}_{\text{XSalsa20-Poly1305}}^{\text{AE}}.$$

Consequence. In the absence of compromise of σ_{prf} , the best available strategy collapses to online guessing, whence

$$\text{Adv}^{\text{dict}} \leq \text{Adv}_{\text{CDH}}^{\text{Ristretto255}} + \text{Adv}_{\text{XSalsa20-Poly1305}}^{\text{AE}} + \Pr[\text{pwd} \in \mathcal{Q}_{\text{on}}].$$

Case 2: σ_{prf} compromised. Then each dictionary guess pwd' can be checked offline by computing y'_{prf} , then x'_{ksf} , $\text{salt}'_{\text{ksf}}$, and finally one attempt to open the envelope. The cost of a single guess is dominated, both asymptotically and empirically, by Argon2id, that is, by the derivation in (9):

$$T_{\text{guess}} \approx T_{\text{Argon2id}}, \quad C_{\text{offline}} \approx |\mathcal{D}| \cdot T_{\text{guess}}.$$

For a ranked password distribution and a fixed compute budget C_{cpu} , the number of tested candidates is bounded by the lower of the compute and memory bottlenecks:

$$G \leq \min \left(\left\lfloor \frac{C_{\text{cpu}}}{T_{\text{guess}}} \right\rfloor, \left\lfloor \frac{B_{\text{mem}}}{256 \text{ MiB}} \right\rfloor \cdot \left\lfloor \frac{T_{\text{wall}}}{T_{\text{guess}}} \right\rfloor \right),$$

where B_{mem} is the memory available to parallel guesses and T_{wall} is the wall-clock duration of the attack window. This calculation is intentionally operational rather than cryptographic: once σ_{prf} is lost, protection shifts from the aPAKE record property to password entropy, KSF cost, rate controls, and incident response.

Scoped security argument 5.1 (Scoped forward secrecy). Within the compromise model stated above, compromise of the long-term keys sk_C and sk_S after session completion is argued not to suffice to recover K_s , under the hardness of CDH and the IND-CCA security of ML-KEM-768.

Proof sketch. After compromise of both long-term keys, the terms containing a static secret can be reconstructed from the public transcript: $dh_1 = sk_S \cdot PK_C$, $dh_2 = sk_S \cdot E_C$, and $dh_3 = sk_C \cdot E_S$. The remaining classical term $dh_4 = e_S \cdot E_C = e_C \cdot E_S$ still requires recovery of a session ephemeral scalar or a break of CDH on the recorded ephemeral points. In parallel, ss_{kem} remains protected under the IND-CCA security of ML-KEM-768 together with correct erasure and absence of leakage from the session-local sk_{kem} .

Thus post-session exposure of long-term keys alone does not reconstruct the full input to the hybrid extractor:

$$\text{Adv}^{\text{FS}} \leq \min\{\text{Adv}_{\text{CDH}}^{\text{Ristretto255}}, \text{Adv}_{\text{ML-KEM}}^{\text{IND-CCA}}\} + \text{Adv}_{\text{HKDF}}^{\text{ext}}. \quad (21)$$

Scoped security argument 5.2 (MAC-confirmation argument). Under the PRF model for HMAC-SHA-512, successful confirmation-token forgery after q_{mac} verification attempts is modeled by the estimate

$$\text{Adv}^{\text{mac-conf}} \leq \text{Adv}_{\text{HMAC}}^{\text{PRF}} + q_{\text{mac}}/2^{512}.$$

Interpretive claim on the hybrid-source combiner. Let

$$\begin{aligned} \text{IKM}_{\text{cl}} &= \text{Encode}(dh_1) \parallel \text{Encode}(dh_2) \\ &\quad \parallel \text{Encode}(dh_3) \parallel \text{Encode}(dh_4), \\ \text{IKM}_{\text{pq}} &= ss_{\text{kem}}. \end{aligned}$$

If, for a fixed transcript τ , at least one of the sources IKM_{cl} or IKM_{pq} remains computationally hidden and retains sufficient entropy for the extractor model, then the extracted pre-key is

$$\text{PRK} = \text{HKDF-Extract}(\text{salt}_{\tau}, \text{IKM}_{\text{cl}} \parallel \text{IKM}_{\text{pq}}).$$

This value is treated as pseudorandom under the standard extractor interpretation of HKDF-Extract. This gives the intended hybrid-source reading of the key schedule: explicit recovery of the real session key from the transcript requires compromise of *both* sources. The statement is not a standalone computational proof of the combiner. It is an extractor-based interpretation that is used consistently in the subsequent symbolic model and implementation checks.

Proof sketch. Let S denote the source among $\{\text{IKM}_{\text{cl}}, \text{IKM}_{\text{pq}}\}$ that remains computationally hidden relative to the fixed transcript τ and retains sufficient entropy in the extractor model, and let X denote the other source. Then $\text{IKM}_{\text{cl}} \parallel \text{IKM}_{\text{pq}}$ may be written as a concatenation of S and X . The standard extractor property of HMAC-based HKDF-Extract [34] suggests that if S is hidden from the adversary with sufficient entropy, then PRK remains pseudorandom even for arbitrary X . This yields the upper bound: indistinguishability of the final key can fail only through a break of the surviving source or a break of the extractor itself:

$$\text{Adv}^{\text{hybrid}} \leq \min\{\text{Adv}_{\text{CDH}}^{\text{Ristretto255}}, \text{Adv}_{\text{ML-KEM}}^{\text{IND-CCA}}\} + \text{Adv}_{\text{HKDF}}^{\text{ext}}. \quad (22)$$

This bound captures the hybrid-source reading carried through the rest of the paper. When at least one of the two sources stays computationally hidden and HKDF-Extract behaves as an extractor, the derived PRK stays pseudorandom up to the additive extractor term. The reading is narrow. Across this reduction sketch and the symbolic analysis below, we model compromise of the final key as something that demands simultaneous weakening of the classical and post-quantum layers. The bound’s right-hand side matches the forward-secrecy bound in Eq. (21), since both encode the same hybrid-source condition, namely that recovering the session key means breaking whichever source survives, classical or post-quantum, beyond the additive extractor term. Figure 5 visualizes this interpretation.

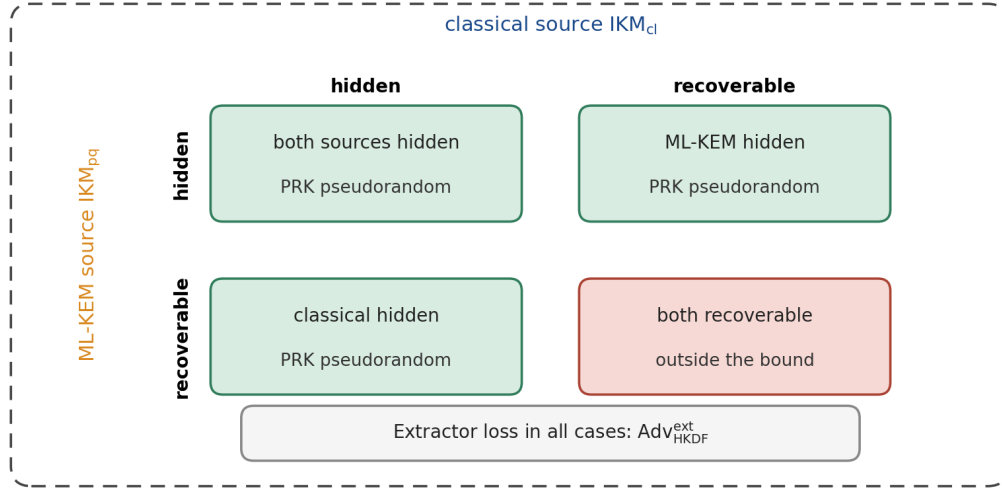


Figure 5: Condition map for the hybrid-combiner interpretation used in Eq. (22). Rows and columns describe whether the classical or ML-KEM source remains hidden relative to a fixed transcript. Under the extractor interpretation, the PRK remains pseudorandom whenever at least one source remains hidden. Simultaneous recovery of both sources is the source-level condition not ruled out by the bound, up to the additive extractor term.

5.2. Resistance to Known Attacks

Offline dictionary attacks. The protection claim is stated for *database compromise without compromise of σ_{opr}* . Under this condition three independent layers operate. (1) The OPRF blocks progress, because without the responder-side OPRF key k_{opr} the adversary cannot compute rwd . (2) Argon2id with MODERATE parameters (256 MiB, 3 iterations) sets

the per-guess cost, where one guess takes roughly $T_{\text{guess}} \approx 0.84$ s on the benchmarking platform, so a sequential exhaustive dictionary search scales as $C_{\text{offline}} \approx |\mathcal{D}| \cdot T_{\text{guess}}$. (3) XSalsa20-Poly1305 forces verification of a candidate to pass authenticated decryption. If $\sigma_{\text{opr}}f$ is compromised, this claim does not automatically transfer to the new threat model and requires separate analysis.

Man-in-the-middle attack. The transcript τ carries the listed public protocol elements, among them pk_{kem} and ct_{kem} . Altering any transcript-bound field produces a MAC mismatch. Other malformed fields fail earlier, during parsing, OPRF or envelope processing, or key confirmation.

Replay attack. Ephemeral nonces (192 bits) and ephemeral keys are generated afresh for every session. For N sessions using uniformly sampled 192-bit nonces, the birthday-bound collision probability is approximately $N(N - 1)/2^{193}$.

Harvest-now-decrypt-later attack. If the classical $dh_1, \dots, dh_4$ terms are later recovered by a quantum adversary, the hybrid key still depends on ss_{kem} , whose protection is tied to ML-KEM-768 and the Module-LWE assumption.

Long-term responder-key exposure. Compromise of sk_S lets an attacker impersonate the record-holding responder in later sessions and makes the static-responder DH terms computable from a recorded transcript, dh_1 and dh_2 among them. Our narrower post-session claim is that sk_S on its own does not reconstruct a completed hybrid session. It does not reveal the initiator static secret, the responder ephemeral scalar, the ephemeral-ephemeral term dh_4 , or the ML-KEM shared secret ss_{kem} , which depends on the destroyed sk_{kem} . The protection statement is therefore scoped to recovery of the recorded hybrid session key, not to impersonation once the responder key is lost.

Timing and microarchitectural side channels. MAC comparisons are performed by constant-time bitwise comparison without early exit, and secrets are zeroized on both successful and rejected paths. The implementation relies on constant-time-oriented Ristretto255 and ML-KEM components, but the evidence here does not include cache-trace, branch-prediction, power, fault-injection, or compiler-level leakage analysis. Side-channel resistance should therefore be treated as an engineering requirement for a hardened deployment, not as a theorem established by the symbolic models.

5.3. Formal Verification

5.3.1. Verification Overview

We analyze the protocol with two symbolic tools in the Dolev–Yao model, ProVerif 2.05 and Tamarin Prover 1.10.0. The aim is to check the structure of the protocol argument, not to certify the reference implementation line by line. We run ProVerif in separate secrecy and authentication models and apply Tamarin to a verified surrogate 4DH model. The results are therefore read together with the modeling boundaries [35, 36] and with the executable artifact that exercises serialization, state handling, and external-call behavior.

Table 10: Symbolic verification summary and modeling boundary

Property	ProVerif	Tamarin	Boundary
Session-key secrecy (P1)	confirmed	confirmed in 27 steps	Symbolic secrecy only. No side-channel or compiled-code claim.
Password secrecy on the wire (P2)	confirmed	confirmed in 3 steps	Password absence is checked in the model trace abstraction.
Classical forward secrecy (P3)	not modeled	confirmed in 119 steps	Uses the surrogate 4DH representation.
Post-quantum contribution (P4)	not modeled separately	represented through the hybrid-source lemma	Does not constitute an IND-CCA proof of ML-KEM.
Initiator authentication (P5a)	confirmed	confirmed in 32 steps	Correspondence assertion under the modeled event structure.
Responder authentication (P5b)	confirmed	confirmed in 17 steps	Correspondence assertion under the modeled event structure.
Injective mutual authentication (P5c)	confirmed	follows from authentication lemmas	Not a proof about API misuse or error formatting.
Hybrid-source interpretation (P6)	not modeled	supported in 29 steps	Evidence for the stated abstraction, not a computational combiner theorem.
Offline guessing in the DB model (P7)	not modeled	supported in 4 steps	Assumes σ_{prf} remains outside the compromised store.
Executability (correctness)	not applicable	confirmed in 16 steps	Confirms existence of an honest completion trace.
Total	5/5 queries	8/8 lemmas	Results belong to separate symbolic abstractions.

Tables 10 and 11 pull apart evidence that often gets conflated. ProVerif is most useful for message logic and correspondence properties in a simpli-

Table 11: Executable artifact checks and what they do not prove

Test area	Representative checks	Does not prove
Wire format and versioning	Exact KE1/KE2/KE3 lengths, unsupported-version rejection, malformed-length rejection	Correctness of every future serialization profile.
Transcript binding	Mutation of transcript-bound fields, including post-quantum material and account context	A general composition theorem for all hybrid PAKEs.
State lifecycle	State consumption, replay rejection, failure-state invalidation, zeroization boundaries	Absence of all memory-forensics or side-channel leakage.
External adapter contract	Public error categories, buffer ownership, safe release paths	Correct integration by arbitrary foreign callers.
Benchmark harnesses	Primitive and protocol workloads under Criterion	Universal timing behavior across platforms.
Total test outcome	157 passed, 1 intentionally ignored slow regression	Machine-checked implementation correctness.

fied unbounded-session setting. Tamarin records stateful properties under the surrogate 4DH abstraction. The executable test suite then checks concrete state-machine behavior that symbolic tools abstract away by design, including wire-format parsing, version and length rejection, transcript-field mutation, replay rejection, failure-state consumption, zeroization boundaries, and external-adapter error contracts. These tests do not replace symbolic analysis. They bind the formalized protocol structure to the implementation behavior that callers actually exercise.

5.3.2. *ProVerif 2.05*

The toolchain is ProVerif 2.05 with OCaml 5.4.0. We use two complementary models, a baseline secrecy model and a separate authentication model, both archived in the accompanying artifact package. The split is intentional, because a single saturated model did not converge stably for all queries at once. The two ProVerif models give compatible but distinct views of the same protocol. Table 12 summarizes them and their principal abstractions.

Table 12: Split ProVerif models and their abstractions

Model	Queries checked	Principal abstraction
Secrecy model	Session-key secrecy and password secrecy on the wire	Authentication events are simplified so that secrecy saturation converges reliably across unbounded sessions.
Authentication model	Initiator, responder, and injective mutual correspondence	Key-material secrecy is abstracted to focus the model on event ordering and agreement.

The split is defensible only because both models share the same message roles, transcript fields, compromise events, and success/failure events. Adding their outcomes as though they formed a single monolithic proof would mislead. They are two consistency checks over the same protocol description.

The most informative query for mutual authentication is P5c, injective mutual correspondence. It states that each successful completion on the responder side matches a distinct completion on the initiator side for the same pair of parties and the same session key. A positive result means that a successful responder-side completion cannot be explained by replaying one successful trace across multiple responder completions. This result is not a computational proof of the implementation. The model omits side channels, variation in error formatting, timing-related leakage, and a range of network-level details that stay within the domain of implementation testing.

5.3.3. *Tamarin Prover 1.10.0*

The toolchain is Tamarin 1.10.0 with Maude 3.4. We verified a surrogate model of the hybrid combiner, and the archived 4DH proof log records a total solving time of **53.84 seconds** for all eight lemmas. Table 13 lists the verified lemmas and their main security interpretation.

The lemma labels follow the property numbering of Table 10, where P4 (the post-quantum contribution) is covered jointly through P6, while injective mutual correspondence (P5c) is established on the ProVerif side, so no lemma index is missing.

The verified Tamarin model does not use an embedded Diffie–Hellman equational theory for the full 4DH construction. It instead replaces 4DH with an abstract secret $\xi_{4dh} = h(\langle sk_C, sk_S, e_C, e_S \rangle)$ and checks which compromise events the adversary needs to derive the session key. This abstraction makes automated reasoning feasible and reads as surrogate symbolic evidence rather than a formal proof of every concrete Ristretto255 operation. The model does not test algebraic independence among $dh_1, \dots, dh_4$, canonical point validation, cofactor-related issues, or equality relationships induced by the group law. Those properties are handled by primitive choice, implementation checks, and the reduction sketches, not by the Tamarin lemma itself. The post-quantum source is modeled in the same surrogate style, as a fresh secret that the adversary recovers only by disclosing the session-ephemeral material, which in the model exposes the classical ephemeral scalars and the ephemeral KEM secret together. This is what lets Lemma P6 express the and-condition, where recovering the session key needs both the classical and

Table 13: Tamarin lemmas established for the surrogate 4DH model

Lemma	Steps	Key property
Session-key secrecy (P1)	27	Without corruption events, the adversary cannot derive the session key
Password secrecy (P2)	3	The password never appears in externally observable model events
Forward secrecy (P3)	119	The session key remains non-derivable after post-session long-term key compromise, assuming session ephemerals are not disclosed
Initiator authentication (P5a)	32	Successful initiator completion requires prior responder participation
Responder authentication (P5b)	17	Successful responder completion requires a valid KE3
Hybrid-source interpretation (P6)	29	Without compromise of the ephemeral secrets, the 4DH and KEM components are not recovered jointly
Offline-guessing resistance in the DB model (P7)	4	Database compromise reveals the record but not the OPRF key
Executability (correctness)	16	A successful protocol-completion trace exists

the post-quantum ephemerals.

Lemma P6 anchors the combiner interpretation. It states that once the session key is established and neither the initiator’s nor the responder’s ephemeral secrets are disclosed, the adversary cannot derive that key. Machine verification of the lemma completes positively in 29 steps.

The lemma permits compromise of long-term keys but disallows disclosure of ephemeral ones. Even with knowledge of the parties’ long-term secrets, the adversary cannot recover the modeled hybrid key material without access to the ephemeral components represented in the classical and post-quantum layers. Within this surrogate model, the result supports the stated hybrid-source interpretation. It does not establish the same claim for a full equational model of Ristretto255 or replace a separate formalization of post-quantum forward secrecy.

5.3.4. *Limits of Interpretation*

To avoid overstating the formal results, we record four principal limitations.

1. **Surrogate representation of 4DH.** The verified Tamarin model ab-

stracts the 4DH component into a single knowledge function. This facilitates the proof but does not replace analysis of the concrete algebraic properties of group operations, canonical point decoding, or independence of the four Diffie–Hellman terms.

2. **Split ProVerif models.** Secrecy and authentication are studied not in one universal model but in two separate artifacts. Hence the 5/5 outcome should be interpreted as the aggregate of two mutually consistent models rather than as a monolithic proof.
3. **Boundary of offline-resistance claims.** Lemma P7 and the corresponding executable tests cover database compromise *without* compromise of σ_{prf} . If σ_{prf} is exposed, the offline-resistance property requires separate analysis and should not be transferred automatically from the model.
4. **Gap between model and implementation.** Symbolic verification does not cover side channels, compiler behavior, memory-allocation properties, external-API mistakes, or vulnerabilities in external crates. For this reason, the formal results reported here complement tests and benchmarks rather than substituting for them.

6. Implementation and Experimental Evaluation

6.1. Architecture of the Reference Implementation

We implement the protocol in **Rust 1.93.1** as a multi-crate workspace. The choice of Rust rests on compile-time memory safety, on a cryptographic core that we keep free of `unsafe` blocks, and on mature mechanisms for secret zeroization. Table 14 reports the implementation surface that backs the evaluation.

6.2. Specification-to-Implementation Traceability

We organize the implementation so that the main protocol statements in the paper can be checked against concrete artifacts rather than against informal intent. Such traceability does not prove correctness, but it lowers the risk that the specification, the implementation, the benchmarks, and the adapter documentation quietly drift into describing different protocols. Table 15 records the correspondence we rely on in the artifact.

The point here is methodological. The symbolic models describe the message structure and the compromise assumptions. The executable tests

Table 14: Source-code metrics of the workspace

Component	Role	LoC	Principal dependencies
Cryptographic core	Primitives, key derivation, and protocol serialization	1513	Ristretto255, ML-KEM, Argon2id
Initiator pipeline	Initiator protocol phases	900	cryptographic core
Responder pipeline	Record-holding responder phases and session completion	907	cryptographic core
Interlanguage layer	Foreign-call adapter and protocol invocation	2026	initiator and responder pipelines
Total (main code)		5346	
Tests		4116	property-based and regression scenarios
Benchmarks		1460	Criterion
Formal models		1806	Tamarin, ProVerif

then check whether the running state machine honors the same message sizes, labels, version handling, and rejection behavior. The adapter documentation, in turn, exposes the subset of those contracts that external callers must obey. This separation keeps the artifact directly auditable, because it treats cryptographic claims, implementation invariants, and foreign-call obligations as related evidence rather than as interchangeable evidence.

6.3. Memory Safety and Secret Zeroization

In a cryptographic system, security rests not only on the chosen primitives but also on the discipline with which secrets are handled in process memory. Here the cryptographic core and both protocol pipelines compile under a strict no-`unsafe`-code policy, while the operations that require low-level pointer manipulation stay confined to a narrow foreign-call boundary. This separation does not replace formal analysis of the implementation. What it does is shrink the defect surface at the memory-management boundary,

Table 15: Traceability between the paper specification and executable artifact

Specification element	Implementation locus	Artifact check
Wire-format lengths and version prefix	Core type constants and protocol serialization/parsing routines	Unit tests reject short, long, and unsupported-version messages. Public length accessors expose the same 1273/1377/65-byte authentication sizes.
Account-bound OPRF derivation	OPRF and registration/envelope modules shared by the initiator and responder crates	OPRF, envelope, and registration tests cover account-specific derivation, validation failures, and rejection of malformed public inputs.
Hybrid 4DH + ML-KEM key path	Initiator authentication, responder authentication, and ML-KEM helper module	Integration and property-oriented tests exercise successful completion, tampered KE2/KE3 rejection, replay rejection, and independent session-key agreement.
Transcript binding of PQ material and acct	Authentication-state construction on both endpoints	Negative tests mutate transcript-bound fields and require authentication failure rather than silent divergence.
State lifetime and zeroization boundary	Initiator state, responder state, and foreign-call lifecycle wrappers	Regression tests check state consumption, rejection after invalid transitions, and safe release paths across the foreign-call boundary.
Public external error contract	Initiator and responder adapter modules plus the public adapter documentation	Adapter lifecycle tests verify that unsupported protocol versions remain distinguishable at the foreign-call boundary instead of being collapsed into generic authentication failure.

where use-after-free defects, double release, loss of correct buffer ownership, and violations of buffer-length invariants would otherwise arise.

Control over the lifetime of a secret matters just as much. The implementation applies automatic zeroization when objects leave scope, backed by dedicated wrappers for transient values, so that sensitive intermediate quantities stay in memory only for the minimal interval the protocol requires. This does not eliminate every possible leak, but it reduces the post-compromise residue of sensitive data in memory dumps, crash logs, and reused buffers. The values we zeroize are the following:

- the user password and the OPRF blinding scalar after OPRF finalization;
- the ephemeral ML-KEM secret after decapsulation;
- the classical 4DH material, the transcript hash, and the PRK after derivation of descendant keys;
- the account-context hash η after completion of KE2/KE3;

- the initiator and responder state structures upon interruption or any rejected completion path.

6.4. *Testing and Reproducibility*

The evaluated revision of the workspace contains 157 passing tests and one slow regression scenario that we keep ignored on purpose. These tests are unit, integration, property-based, and regression checks that cover the protocol lifecycle, the wire format, the external API contract, OPRF/KEM correctness, and secure rejection of invalid states.

For archival review, the source snapshot should record the exact commit identifier along with the commands that reproduce each evidence layer. Those commands are `cargo test -workspace` for the executable tests, the documented Criterion benchmark targets for timing, `proverif` for the two ProVerif files, and `tamarin-prover` for the archived Tamarin model. For the snapshot described here, a faithful review should yield 157 passed tests, one slow regression that stays ignored on purpose, 5/5 ProVerif queries, and 8/8 Tamarin lemmas.

These checks complement the symbolic analysis rather than duplicate it. The executable tests exercise serialization, version handling, state-lifecycle invariants, and input-boundary conditions in the running implementation. The Criterion 0.5.1 benchmarks [37] quantify the cost of the main primitives and protocol phases. The archived ProVerif and Tamarin models record the formal abstractions on which the security claims rest.

We ran the benchmarks on a Linux server with two Intel Xeon E5-2650 v4 processors (24 physical cores, 48 hardware threads, 60 MiB aggregate L3 cache, Ubuntu kernel 6.8.0-111, Rust 1.93.1) in deterministic optimized-build mode. Criterion estimates the mean and the 95% confidence interval by bootstrap, so the values reported below should be read as reproducible estimates for this platform and not as universal constants for every environment. Re-running the full test suite, the benchmark suite, and the formal models gives an independent check on the main quantitative and structural claims we report here.

6.5. *Benchmarks of Cryptographic Primitives*

Figure 6 summarizes the scale separation before the detailed benchmark tables.

Table 16 gives the primitive-level measurements behind this profile.

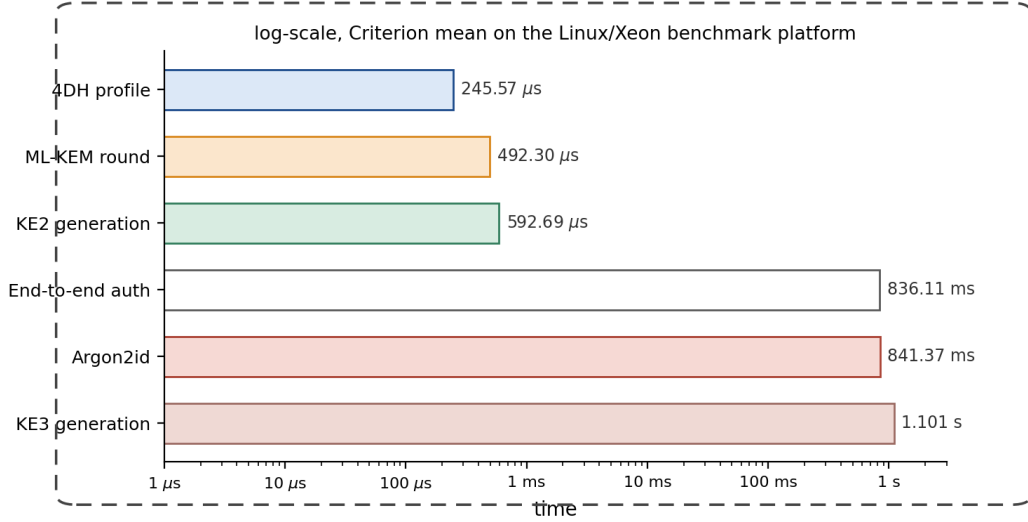


Figure 6: Runtime profile on the Linux/Xeon benchmark platform. The password-hardening step dominates end-to-end latency, while the post-quantum and classical key-exchange components remain sub-millisecond.

Table 16: Benchmark results for cryptographic primitives

Primitive	Operation	$\hat{\mu}$	95% CI
Ristretto255	Key-pair generation	41.85 μ s	[39.47; 44.28]
Ristretto255	Single DH	75.93 μ s	[74.11; 77.90]
Ristretto255	3DH (baseline)	262.54 μ s	[258.50; 266.58]
Ristretto255	4DH (protocol profile)	245.57 μs	[242.98; 249.55]
ML-KEM-768	Key-pair generation	151.48 μ s	[147.39; 155.63]
ML-KEM-768	Encapsulation	146.50 μ s	[142.58; 150.51]
ML-KEM-768	Decapsulation	172.74 μ s	[168.74; 176.75]
ML-KEM-768	Full round	492.30 μs	[479.59; 505.34]
OPRF	Blinding	115.63 μ s	[113.63; 117.64]
OPRF	Responder evaluation	76.39 μ s	[74.95; 77.85]
OPRF	Finalization	109.11 μ s	[107.29; 110.91]
HKDF	Extract	3.63 μ s	[3.52; 3.74]
HKDF	Expand (64 bytes)	2.26 μ s	[2.20; 2.33]
HMAC-SHA-512	256 bytes	2.92 μ s	[2.90; 2.94]
XSalsa20-Poly1305	Encrypt 96 bytes	2.52 μ s	[2.49; 2.57]
XSalsa20-Poly1305	Decrypt 96 bytes	2.67 μ s	[2.66; 2.69]
Argon2id	MODERATE	841.37 ms	[830.54; 851.96]
Hybrid combiner	HKDF-Extract (160 bytes)	2.49 μ s	[2.47; 2.50]

The micro-benchmarks show three stable relationships. First, the full ML-KEM-768 round (492.30 μ s) and the measured 4DH protocol profile (245.57 μ s) remain sub-millisecond operations, so the post-quantum layer does not alter the computational class of the protocol. Second, Argon2id (841.37 ms in isolated measurement) dominates all other components and determines the perceived authentication latency. Third, the hybrid key-extraction step (2.49 μ s) is negligible relative to the KSF and does not constitute a separate bottleneck.

6.6. Protocol-Level Benchmarks

The phase measurements and the complete authentication measurement are separate Criterion workloads with different state-initialization and batching boundaries. They are therefore not additive. Table 17 reports scenario-level timings rather than a decomposition whose rows sum to the end-to-end value.

Table 17: Benchmark results for protocol-level scenarios, reported as separate Criterion workloads that are not additive across rows

Phase	Measurement scenario	$\hat{\mu}$	95% CI	Main cost
Registration	Registration-request creation	127.37 μ s	[125.24; 130.09]	OPRF blinding
Registration	Responder response creation	71.42 μ s	[70.98; 71.84]	OPRF evaluation
Registration	Registration completion	1.065 s	[993.93 ms; 1.143 s]	Argon2id
Authentication	KE1 generation	304.87 μ s	[288.05; 322.18]	OPRF + ML-KEM keygen
Authentication	KE2 generation	592.69 μ s	[582.16; 606.72]	4DH + ML-KEM encapsulation
Authentication	KE3 generation at the initiator	1.101 s	[1.034; 1.169] s	Argon2id + decapsulation
Authentication	Final responder verification	2.61 μ s	[2.31; 3.06]	MAC verification
Full	End-to-end authentication cycle	836.11 ms	[825.41; 847.99]	Argon2id

Responder KE2 generation remains a sub-millisecond operation, whereas Argon2id contributes the overwhelming share of time.

6.7. Key Quantitative Findings and Implementation Implications

The isolated responder profile shows that the responder’s cryptographic core stays compatible with high-throughput deployment. Isolated KE2 generation takes 592.69 μ s, while final responder verification takes only 2.61 μ s. For a cryptography-bound responder, this implies a theoretical upper bound

of roughly 1.69×10^3 authentications per second per core, once we set aside database access, telemetry, and business logic.

For the complete authentication without the KSF, the post-quantum component adds $379.45 \mu\text{s}$ (+48.3%) of CPU time. Set against the Argon2id background, that excess shrinks to roughly 0.045% of a typical end-to-end scenario. The real trade-off is therefore not CPU cost but network volume. The complete handshake occupies 2715 bytes in wire format, which is 2272 bytes more than the classical variant at the same three-message interaction profile.

Comparative claim relative to classical OPAQUE. Let $M_{\text{opq}}, R_{\text{opq}}, C_{\text{opq}}, W_{\text{opq}}$ denote, respectively, the message count, network round-trip cost, cryptographic computation, and communication volume of canonical 3DH-OPAQUE without the KSF, and let $M_{\text{hyb}}, R_{\text{hyb}}, C_{\text{hyb}}, W_{\text{hyb}}$ denote the corresponding quantities for Hybrid PQ-OPAQUE. Let C_{misc} collect the fixed transcript-binding, serialization, and key-derivation work introduced by the hybrid path. Then

$$\begin{aligned} M_{\text{hyb}} &= M_{\text{opq}} = 3, \\ R_{\text{hyb}} &= R_{\text{opq}} = \frac{3}{2} \text{RTT}, \\ C_{\text{hyb}} &= C_{\text{opq}} + C_{\text{DH}} + C_{\text{KEM}} + C_{\text{misc}}, \\ W_{\text{hyb}} &= W_{\text{opq}} + \Delta W, \quad \Delta W = 2272 \text{ bytes}. \end{aligned}$$

On the benchmark platform, this corresponds to a measured increase of $C_{\text{DH}} + C_{\text{KEM}} + C_{\text{misc}} \approx 379.45 \mu\text{s}$ without the KSF. Compared with the established baseline of classical OPAQUE, the cost for honest parties rises additively, and the interaction profile is unchanged. Within the present threat model, explicit compromise of the final key material is not reducible to weakening the classical contribution alone. It is interpreted as requiring compromise of both the classical and post-quantum layers. Table 18 states the same comparison in structural form.

7. Discussion of Results

7.1. Interpretation of the Formal and Experimental Evidence

The evidence is layered, not monolithic, and each layer answers a different question. ProVerif handles secrecy and correspondence in symbolic models, Tamarin captures stateful properties of the surrogate 4DH model, and the

Table 18: Comparison of Hybrid PQ-OPAQUE with canonical 3DH-OPAQUE

Aspect	3DH-OPAQUE	Hybrid PQ-OPAQUE	Interpretation
Interaction profile	3 messages, 1.5 RTTs	3 messages, 1.5 RTTs	Protocol interaction is unchanged
Computational complexity for honest parties	Baseline 3DH path	+1 additional DH, + KEM, + fixed transcript/KDF work	The increase is additive rather than structural
Communication complexity	Baseline handshake volume	+2272 bytes in wire format	The principal practical trade-off shifts to transport
Key-compromise condition	Weakening of the classical contribution	Compromise of both the classical and PQ layers	The recovery condition is stricter within the present threat model

Rust test suite exercises executable behavior that symbolic tools never see, including serialization, state lifetime, boundary checks, version handling, and the external API contract. All three point to the same protocol description, yet they do not carry the same evidentiary weight.

The weakest link in the argument is the hybrid-source reading of the combiner. The extractor rationale and the surrogate 4DH Tamarin lemma both support it, but it remains short of a full computational proof. That distinction matters, because the paper does not claim that the implementation is machine-verified. What it claims is narrower, that the design, models, tests, and measurements are mutually consistent under the stated assumptions. Table 19 sets each of these claims against its boundary.

Table 19: Scope of the main claims and supporting evidence

Claim	Evidence used in the paper	Boundary of the claim
Fixed-round hybridization	Explicit message syntax, transcript definitions, wire-format accounting, and Rust serialization tests	The construction is OPAQUE-like and does not claim complete RFC 9807 conformance.
Transcript-bound post-quantum contribution	Key-schedule equations, context binding through <code>acct</code> , formal models, and integration tests	The formal tools reason about symbolic abstractions, not the compiled binary or side-channel behavior.
Resistance to database-only offline guessing	OPRF-key separation, database-compromise model, negative regression tests, and the stated trust boundary for σ_{oprf}	The claim does not survive compromise of the OPRF root secret or a deployment that stores it with the password database.
Hybrid-source key dependence	HKDF extractor rationale, the surrogate 4DH Tamarin lemma, and agreement between the specification and implementation paths	This is evidence for the intended combiner interpretation, not a full computational proof of the hybrid PAKE.
Practical overhead is dominated by the KSF	Criterion measurements for primitive costs, end-to-end authentication, and message-size accounting	The absolute timings are platform-specific and must be recalibrated for mobile, embedded, or high-latency deployments.

7.2. Performance, KSF Parameters, and the Trust Boundary

The performance profile is governed almost entirely by Argon2id, while ML-KEM-768 and 4DH stay in the sub-millisecond range. The binding deployment constraint is therefore not the absolute cost of ML-KEM-768 but the KSF profile chosen for the target initiator population. Our selected profile suits web and B2B settings, while mobile and edge deployments may need separate KSF calibration and stricter rate limiting.

The trust boundary around $\sigma_{\text{opr}}f$ is not a cosmetic detail. We claim resistance to offline guessing only for database compromise *without* compromise of that secret. In operational terms $\sigma_{\text{opr}}f$ is the root cryptographic secret of the service, and it demands separated storage, segregated access, minimal memory residency, and a rotation policy that treats OPRF-key derivation as privileged cryptographic state.

One practical advantage of the migration path is that the post-quantum component stays ephemeral and never enlarges the registration record. The work of migration sits in the application-layer authentication logic, version negotiation, and contextual binding of `acct`. Growth of the wire format to 2715 bytes does make transport constraints more consequential than in classical OPAQUE. For TCP/QUIC this is largely a buffering issue, whereas datagram deployments also have to contend with fragmentation and MTU concerns.

7.3. Migration, Compatibility, and Operational Viability

The construction can be deployed without redesigning the entire identity stack, for two reasons. First, the registration record accumulates no long-lived post-quantum material, since ML-KEM appears only in the ephemeral authentication path. Second, explicit wire-format versioning and contextual binding of `acct` allow a staged rollout that leaves existing user state intact. For operators, then, migration looks more like controlled protocol evolution than wholesale credential re-issuance.

Operational compatibility matters just as much. Authentication interacts with login-attempt limits, logging, account recovery, anti-fraud analysis, and audit requirements. For that reason the implementation emphasizes deterministic serialization, explicit version checking, and stable external-call boundaries. Failure modes should stay distinguishable, since cryptographic failure, version-negotiation failure, and transport failure carry different operational meanings, and folding them into a generic failed login would obscure diagnosis during deployment. The same separation between crypto-

graphic, transport, and version-negotiation failures matters in audit-heavy cyber-physical deployments, where authentication is one stage of a broader telemetry and inspection pipeline rather than an isolated login step, as noted in the application motivation in Section 1.

7.4. Reproducibility and Artifact Transparency

The artifact is valuable because it forces the specification to meet the implementation boundary. The same wire sizes, transcript fields, key schedule, and error paths recur across the paper, the Rust crates, and the adapter documentation. A reader can therefore check the written protocol against executable behavior instead of taking the paper and the implementation as two separate claims.

This linkage also keeps our reading of the evidence disciplined. The hybrid-source argument belongs to the Tamarin abstraction and the combiner rationale. Parsing correctness, rejection of non-canonical points, version handling, and prevention of state reuse are properties of the reference implementation and its tests. For post-quantum migration work, that separation pays off, because it exposes the strengths of the artifact without hiding the parts that still need stronger analysis.

7.5. Validity, Limitations, and Future Directions

The work carries several material limitations. The implementation is OPAQUE-like rather than fully conformant to RFC 9807 and the associated standard interfaces [5]. The symbolic models do not replace a computational proof of the implementation code, and the benchmark measurements come from a single hardware platform, so absolute timing values should be carried over to other architectures with caution. Finally, ML-KEM-768 is a recent standard, and hybridization reduces dependence on any single assumption without removing the need for continued cryptanalysis of the post-quantum layer.

We see four natural directions for extending the work:

1. adaptation to RFC 9807 and the associated OPAQUE/OPRF ecosystem;
2. formalization of the hybrid extension in a computational or UC setting [27];
3. investigation of alternative KSF profiles and ML-KEM variants for mobile and IoT scenarios;

4. integration of external post-quantum signatures for settings in which non-repudiation is required, without altering the baseline HMAC-based handshake confirmation.

8. Conclusion

Numerical result. We specify an OPAQUE-derived hybrid authentication profile that adds ML-KEM-768 to the authenticated transcript while leaving the KE1-KE2-KE3 interaction pattern unchanged. The complete hybrid handshake occupies 2715 bytes on the wire, a 2272-byte expansion over the classical format. On the stated Linux/Xeon platform, the full authentication workload runs in 836.11 ms, the isolated ML-KEM-768 round in 492.30 μ s, the 4DH protocol profile in 245.57 μ s, and the KSF-free post-quantum path adds 379.45 μ s. Read together, these numbers locate the cost where it actually falls. Argon2id dominates latency, whereas ML-KEM mainly changes the transport budget.

Validated boundary. The evidence supports a bounded design argument rather than a universal security claim. ProVerif confirms the modeled secrecy and authentication queries in two separate models, Tamarin establishes 8/8 lemmas in a surrogate 4DH model, and the reference implementation covers the wire format, the state machine, rejection behavior, tests, and benchmark paths. The limitations matter just as much. The implementation is not fully RFC 9807 conformant, the Tamarin model abstracts the concrete Ristretto255 algebra, the ProVerif strategy is split across two files, the benchmark uses one hardware platform, and dictionary resistance is claimed only for database compromise without compromise of $\sigma_{\text{opr}}f$.

Next work. The natural next step is a conformance-oriented profile aligned more tightly with RFC 9807 and the CFRG OPRF ecosystem [24], after which a computational combiner proof or a UC treatment of the hybrid extension becomes the priority. Wider hardware measurements remain necessary for mobile, browser, and embedded deployments. A hardened deployment profile should then add side-channel evaluation, operational handling of $\sigma_{\text{opr}}f$, and a migration plan for environments that require post-quantum signatures.

Data and Software Availability

The implementation, tests, benchmarks, and formal models are maintained in the accompanying source repository: <https://github.com/>

[oleksandrmelnynchenko/aura-security-opaque-rs](https://github.com/oleksandrmelnynchenko/aura-security-opaque-rs). The submission artifact should identify the exact revision used for review and, where required by the venue, include the same revision as a source bundle or DOI-backed archive. The snapshot contains the Rust protocol crates, foreign-call adapter, deterministic and property-based tests, Criterion benchmark harnesses, ProVerif models, Tamarin model, and archived verification logs. For the artifact state described here, the reproducibility target is 157 passed tests, one intentionally ignored slow regression, 5/5 ProVerif queries, and 8/8 Tamarin lemmas.

Funding

The author declares that this research received no external funding.

CRedit authorship contribution statement

Oleksandr Melnychenko: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of Competing Interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of Generative AI and AI-assisted technologies in the writing process

OpenAI Codex was used during manuscript preparation to assist with English-language editing, consistency checks, and minor $\text{\LaTeX}$ typesetting support in portions of the abstract, introduction, discussion, conclusion, and connective prose. OpenAI Codex was not used as an authoritative source for technical claims, references, equations, implementation results, or experimental measurements. All protocol descriptions, equations, references, tables, implementation details, and experimental results were independently reviewed, verified, and approved by the author, who takes full responsibility for the content of the article.

References

- [1] Bonneau J., Herley C., van Oorschot P.C., Stajano F. The quest to replace passwords: a framework for comparative evaluation of web authentication schemes. In: Proceedings of the IEEE Symposium on Security and Privacy, 2012, pp. 553–567. DOI: <https://doi.org/10.1109/SP.2012.44>.
- [2] Florencio D., Herley C. A large-scale study of web password habits. In: Proceedings of the International Conference on World Wide Web, 2007, pp. 657–666. URL: <https://archives.iw3c2.org/www2007/papers/paper620.pdf>.
- [3] Bellare M., Pointcheval D., Rogaway P. Authenticated key exchange secure against dictionary attacks. In: EUROCRYPT 2000, LNCS 1807, Springer, 2000, pp. 139–155. DOI: https://doi.org/10.1007/3-540-45539-6_11.
- [4] Jarecki S., Krawczyk H., Xu J. OPAQUE: an asymmetric PAKE protocol secure against pre-computation attacks. In: EUROCRYPT 2018, LNCS 10822, Springer, 2018, pp. 456–486. DOI: https://doi.org/10.1007/978-3-319-78372-7_15.
- [5] Bourdrez D., Krawczyk H., Lewi K., Wood C.A. The OPAQUE Augmented Password-Authenticated Key Exchange (aPAKE) Protocol. IRTF RFC 9807, July 2025. URL: <https://www.rfc-editor.org/info/rfc9807>.
- [6] Shor P.W. Algorithms for quantum computation: discrete logarithms and factoring. In: Proceedings of the IEEE Symposium on Foundations of Computer Science, 1994, pp. 124–134. DOI: <https://doi.org/10.1109/SFCS.1994.365700>.
- [7] Bernstein D.J., Lange T. Post-quantum cryptography. Nature 549(7671) (2017) 188–194. DOI: <https://doi.org/10.1038/nature23461>.
- [8] Roetteler M., Naehrig M., Svore K.M., Lauter K. Quantum resource estimates for computing elliptic curve discrete logarithms. In: ASIACRYPT 2017, LNCS 10625, Springer, 2017, pp. 241–270. DOI: https://doi.org/10.1007/978-3-319-70697-9_9.

- [9] Babbush R., Zaleman A., Gidney C., Broughton M., Khattar T., Neven H., Bergamaschi T., Drake J., Boneh D. Securing elliptic curve cryptocurrencies against quantum vulnerabilities: resource estimates and mitigations. Google Quantum AI whitepaper, March 30, 2026. URL: <https://quantumai.google/static/site-assets/downloads/cryptocurrency-whitepaper.pdf>.
- [10] Mosca M. Cybersecurity in an era with quantum computers: will we be ready? IEEE Security & Privacy 16(5) (2018) 38–41. DOI: <https://doi.org/10.1109/MSP.2018.3761723>.
- [11] National Institute of Standards and Technology. Module-Lattice-Based Key-Encapsulation Mechanism Standard: FIPS 203. 2024. URL: <https://csrc.nist.gov/pubs/fips/203/final>.
- [12] National Institute of Standards and Technology. Module-Lattice-Based Digital Signature Standard: FIPS 204. 2024. URL: <https://csrc.nist.gov/pubs/fips/204/final>.
- [13] Stebila D., Fluhrer S., Gueron S. Hybrid key exchange in TLS 1.3. IETF Internet-Draft draft-ietf-tls-hybrid-design-16, work in progress, September 2025. URL: <https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/>.
- [14] Brendel J., Fischlin M., Günther F., Janson C., Stebila D. Towards post-quantum security for Signal’s X3DH handshake. In: SAC 2020, LNCS 12804, Springer, 2020, pp. 404–430. DOI: https://doi.org/10.1007/978-3-030-81652-0_16.
- [15] Kret E., Schmidt R. The PQXDH Key Agreement Protocol. Signal, 2023. URL: <https://signal.org/docs/specifications/pqxdh/>.
- [16] Hülsing A., Ning K.-C., Schwabe P., Weber F., Zimmermann P. R. Post-quantum WireGuard. In: Proceedings of the IEEE Symposium on Security and Privacy, 2021, pp. 304–321. DOI: <https://doi.org/10.1109/SP40001.2021.00030>.
- [17] Arriaga A., Barbosa M., Jarecki S., Skrobot M. C’est très CHIC: a compact password-authenticated key exchange from lattice-based KEM. Cryptology ePrint Archive, Paper 2024/308, 2024. URL: <https://eprint.iacr.org/2024/308>.

- [18] Lyu Y., Liu S., Han S. Universal composable password authenticated key exchange for the post-quantum world. In: EUROCRYPT 2024, LNCS 14657, Springer, 2024, pp. 120–150. DOI: https://doi.org/10.1007/978-3-031-58754-2_5.
- [19] Hesse J., Rosenberg M. PAKE combiners and efficient post-quantum instantiations. Cryptology ePrint Archive, Paper 2024/1621, 2024. URL: <https://eprint.iacr.org/2024/1621>.
- [20] Lyu Y., Liu S. Hybrid password authentication key exchange in the UC framework. Cryptology ePrint Archive, Paper 2024/1630, 2024. URL: <https://eprint.iacr.org/2024/1630>.
- [21] Boyko V., MacKenzie P., Patel S. Provably secure password-authenticated key exchange using Diffie–Hellman. In: EUROCRYPT 2000, LNCS 1807, Springer, 2000, pp. 156–171. DOI: https://doi.org/10.1007/3-540-45539-6_12.
- [22] Bellare S.M., Merritt M. Encrypted key exchange: password-based protocols secure against dictionary attacks. In: Proceedings of the IEEE Symposium on Security and Privacy, 1992, pp. 72–84. DOI: <https://doi.org/10.1109/RISP.1992.213269>.
- [23] Ladd W. SPAKE2, a password-authenticated key exchange. IRTF RFC 9382, September 2023. URL: <https://www.rfc-editor.org/info/rfc9382>.
- [24] Davidson A., Faz-Hernández A., Sullivan N., Wood C.A. Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups. IRTF RFC 9497, December 2023. URL: <https://www.rfc-editor.org/info/rfc9497>.
- [25] Haase B., Labrique B. AuCPace: efficient verifier-based PAKE protocol tailored for the IIoT. IACR Transactions on Cryptographic Hardware and Embedded Systems 2019(2) (2019) 1–48. DOI: <https://doi.org/10.46586/tches.v2019.i2.1-48>.
- [26] Wu T. The Secure Remote Password protocol. In: Proceedings of NDSS, 1998, pp. 97–111. URL: <https://www.ndss-symposium.org/ndss1998/secure-remote-password-protocol/>.

- [27] Canetti R. Universally composable security: a new paradigm for cryptographic protocols. In: Proceedings of the IEEE Symposium on Foundations of Computer Science, 2001, pp. 136–145. DOI: <https://doi.org/10.1109/SFCS.2001.959888>.
- [28] de Valence H., Grigg J., Hamburg M., Lovecraft I., Tankersley G., Val-sorda F. The ristretto255 and decaf448 groups. IRTF RFC 9496, December 2023. URL: <https://www.rfc-editor.org/info/rfc9496>.
- [29] Esgin M.F., Steinfeld R., Tairi E., Xu J. LeOPaRd: towards practical post-quantum oblivious PRFs via interactive lattice problems. Cryptology ePrint Archive, Paper 2024/1615, 2024. URL: <https://eprint.iacr.org/2024/1615>.
- [30] Castryck W., Decru T. An efficient key recovery attack on SIDH. In: EUROCRYPT 2023, LNCS 14008, Springer, 2023, pp. 423–447. DOI: https://doi.org/10.1007/978-3-031-30589-4_15.
- [31] Vos J., Jarecki S., Wood C.A. Hybrid Post-Quantum Password Authenticated Key Exchange. Individual Internet-Draft draft-vos-cfrg-pqqake-00, April 2025. URL: <https://datatracker.ietf.org/doc/draft-vos-cfrg-pqqake/00/>.
- [32] Biryukov A., Dinu D., Khovratovich D. Argon2: new generation of memory-hard functions for password hashing. In: Proceedings of IEEE European Symposium on Security and Privacy, 2016, pp. 292–302. DOI: <https://doi.org/10.1109/EuroSP.2016.31>.
- [33] Krawczyk H., Eronen P. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). IETF RFC 5869, 2010. URL: <https://www.rfc-editor.org/info/rfc5869>.
- [34] Krawczyk H. Cryptographic Extraction and Key Derivation: The HKDF Scheme. In: CRYPTO 2010, LNCS 6223, Springer, 2010, pp. 631–648. DOI: https://doi.org/10.1007/978-3-642-14623-7_34.
- [35] Cremers C., Mauw S. Operational Semantics and Verification of Security Protocols. Springer, 2012. DOI: <https://doi.org/10.1007/978-3-540-78636-8>.

- [36] Basin D., Cremers C., Meadows C. Model checking security protocols. In: Handbook of Model Checking. Springer, 2018, pp. 727–762. DOI: https://doi.org/10.1007/978-3-319-10575-8_22.
- [37] Heisler B. Criterion.rs: statistics-driven micro-benchmarking in Rust. Project site. URL: <https://github.com/bheisler/criterion.rs>.
- [38] Svystun S., Scisło L., Pawlik M., Melnychenko O., Radiuk P., Savenko O., Sachenko A. DyTAM: accelerating wind turbine inspections with dynamic UAV trajectory adaptation. Energies 18(7) (2025) 1823. DOI: <https://doi.org/10.3390/en18071823>.
- [39] Svystun S., Melnychenko O., Radiuk P., Savenko O., Sachenko A., Lysyi A. Thermal and RGB images work better together in wind turbine damage detection. International Journal of Computing 23(4) (2024) 526–535. DOI: <https://doi.org/10.47839/ijc.23.4.3752>.
- [40] Lysyi A., Sachenko A., Radiuk P., Lysyi M., Melnychenko O., Ishchuk O., Savenko O. Enhanced fire hazard detection in solar power plants: an integrated UAV, AI, and SCADA-based approach. Radioelectronic and Computer Systems (2) (2025) 99–117. DOI: <https://doi.org/10.32620/reks.2025.2.06>.